

Principia Softwarica: The Plan 9 Profilers

version 0.1

Yoann Padioleau
`yoann.padioleau@gmail.com`

with code from
Ken Thompson and Rob Pike

March 18, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1	Introduction	5
1.1	Motivations	5
1.2	The Plan 9 profilers: <code>prof</code> , <code>tprof</code> , <code>trace</code> , <code>stats</code>	5
1.3	Other profilers	6
1.4	Getting started	6
1.5	Requirements	6
1.6	About this document	6
1.7	Copyright	7
1.8	Acknowledgments	7
2	Overview	8
2.1	Profiler principles	8
2.1.1	User time vs system time vs real time	8
2.1.2	Ticks	8
2.1.3	Timer interrupt and preemptive scheduling	9
2.1.4	Instrumentation profilers	10
2.1.5	Sampling profilers	10
2.1.6	Tracing and monitoring profilers	10
2.2	<code>time</code> command-line interface	10
2.3	<code>prof</code> and <code>tprof</code> command-line interfaces	10
2.4	<code>trace</code> and <code>stats</code> graphical user interfaces	11
2.5	<code>iostats</code> filesystem interface	11
2.6	Profiling <code>hello</code>	12
2.7	Code organization	13
2.8	Software Architecture	14
2.9	Book structure	15
3	Kernel Support	16
3.1	Ticks, HZ, and times	16
3.2	Wait message	16
3.3	<code>/proc/<pid>/status</code> and <code>/dev/cputime</code>	17
3.4	<code>/proc/<pid>/profile</code>	17
3.5	<code>/proc/trace</code>	17
3.6	TOS	18
4	Linker Support	19
4.1	<code>5l -p</code>	19
4.2	<code>5l -p1</code>	20
4.3	The symbol table	20

5	Timing a Command: /bin/time	22
5.1	main()	22
5.2	Error management	23
5.3	Extra display	24
6	Instrumentation-based Profiler: /bin/prof	25
6.1	Data structures	25
6.2	main()	26
6.3	Symbol table loading: <code>syms()</code>	28
6.4	Profiling data loading: <code>datas()</code>	28
6.5	Displaying data: <code>plot()</code>	29
6.6	Call graph, <code>prof -d</code>	32
6.6.1	<code>prof -d -r</code>	34
7	Sampling-based Profiler: /bin/tprof	35
7.1	Example: <code>longrun.c</code>	35
7.2	main()	36
7.3	Reading the symbol table from the binary	37
7.4	Reading the profile data from <code>/proc/<pid>/profile</code>	38
7.5	Displaying profile	38
7.6	Error management	39
8	Profiling the Kernel: /bin/kprof	41
9	Show real-time Behavior: /bin/trace	45
9.1	Data structures	45
9.2	main()	46
10	System Monitoring /bin/stats	58
10.1	Data structures	58
10.2	main()	62
11	IO Monitoring: /bin/iostats	65
12	Conclusion	66
12.1	Connections to other books	66
12.2	Beyond the Plan 9 profilers	66
A	Extra Code	68
A.1	profilers/	68
A.1.1	misc/time.c	68
A.1.2	misc/kprof.c	68
A.1.3	misc/prof.c	68
A.1.4	misc/trace.c	69
A.1.5	misc/tprof.c	72
A.1.6	misc/stats.c	73
A.2	iostats/	95
A.2.1	iostats/statfs.h	95
A.2.2	iostats/globals.c	98
A.2.3	iostats/iostats.c	99
A.2.4	iostats/statsrv.c	110

Glossary	124
Index	125
References	125

Chapter 1

Introduction

The goal of this book is to present in full details the source code of a few profilers.

1.1 Motivations

Writing correct code is only half the job—making it fast enough is the other half. A profiler tells you *where* your program actually spends its time, replacing guesswork with measurement. Without a profiler, programmers often optimize the wrong function, or worse, make the code harder to read for no measurable gain. As the saying goes: “premature optimization is the root of all evil” (Knuth), but *informed* optimization, guided by profiling data, is simply good engineering.

Understanding how a profiler works gives you insight into the interplay between user programs, the compiler toolchain, and the kernel. A profiler like `prof` relies on the linker to instrument function calls, while `tprof` relies on the kernel to sample the program counter at regular intervals. Studying these tools therefore ties together concepts from LINKER book [Pad15] and KERNEL book [Pad14].

Here are a few questions I hope this book will answer:

- How do you measure the time spent by a program? What is the difference between user time, system time, and real time (wall-clock time)?
- How does a sampling profiler work? How does the kernel periodically record where a program is executing?
- How does an instrumenting profiler work? What code does the linker insert at function entry and exit to record call counts and timing?
- How can you trace system calls and visualize system-wide activity in real time?
- What is the role of the `/proc` filesystem and the TOS (Top of Stack) structure in supporting profiling on Plan 9?

1.2 The Plan 9 profilers: `prof`, `tprof`, `trace`, `stats`

Unlike most books in Principia Softwarica, which focus on a single program (one assembler, one linker, one shell), this book covers a *collection* of small profiling tools. The reason is that profiling is not a single problem but a spectrum: you might want to time an entire command (`time`), identify which functions are hot (`prof`, `tprof`), trace individual system calls (`trace`), monitor system-wide resource usage (`stats`), or measure the I/O cost of a program (`iostats`). Each tool addresses a different point on that spectrum, and each is small enough—a few hundred lines of C—that together they still form a manageable codebase.

Like for most books in Principia Softwarica, I chose Plan 9 programs because those programs are simple, small, elegant, open source, and they form together a coherent set. The Plan 9 profilers total roughly 3000 lines of code, compared to the tens of thousands of lines in Linux’s `perf` tool alone.

1.3 Other profilers

Here are a few profilers that I considered for this book, but which I ultimately discarded:

- `gprof` [?] is the classic UNIX profiler. It combines instrumentation (the compiler inserts code at each function entry to record call arcs) with statistical sampling (the kernel periodically interrupts the program to record its program counter). The combination gives both call counts and time estimates per function. `gprof` was introduced with 4.2BSD and became the standard profiling tool on UNIX systems for decades. The Plan 9 `prof` tool works on a similar principle but is simpler, relying on the linker rather than the compiler for instrumentation. GNU `gprof`, part of the `binutils` package, is a reimplementaion of the original BSD tool that follows the same design (compiler-inserted call arcs plus PC sampling) and reads the same `gmon.out` file format. The original BSD `gprof` was about 3000 LOC; GNU `gprof` (part of `binutils`) is roughly 15 000 LOC.
- OProfile is a system-wide statistical profiler for Linux that uses hardware performance counters when available. Unlike `gprof`, which requires recompiling the target program, OProfile can profile any running program, including the kernel itself, with low overhead.
- `perf` is the modern Linux profiling framework. It subsumes most of what OProfile and `gprof` do, and adds tracing, hardware counter access, and visualization. However, the `perf` codebase is enormous—tens of thousands of lines of C—making it unsuitable for a teaching book.

Figure 1.1 presents a timeline of major profiling tools. The Plan 9 profilers are small and self-contained, making them ideal for a teaching book. They cover the three main profiling approaches—timing (`time`), instrumentation (`prof`), and sampling (`tprof`)—plus system-wide monitoring (`stats`, `trace`) and I/O tracing (`iostats`), all in a fraction of the code found in modern tools like `perf`.

1.4 Getting started

To get started you first need to get the source of the Plan 9 version described in this book. See <https://www.principia-softwarica.org/getting-started.html>. To use `prof`, you must link your target program with the `-p` flag so that the linker inserts profiling instrumentation. The `tprof` tool instead relies on kernel support (`/proc/<pid>/profile`) and requires no special linking.

1.5 Requirements

This book assumes familiarity with C and with Plan 9’s basic conventions (the `/proc` filesystem, the `Waitmsg` structure, the `rfork` system call). Readers who have gone through KERNEL book [Pad14] will find the kernel-support chapter straightforward; readers who have read LINKER book [Pad15] will immediately understand how 51 `-p` instruments function calls.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

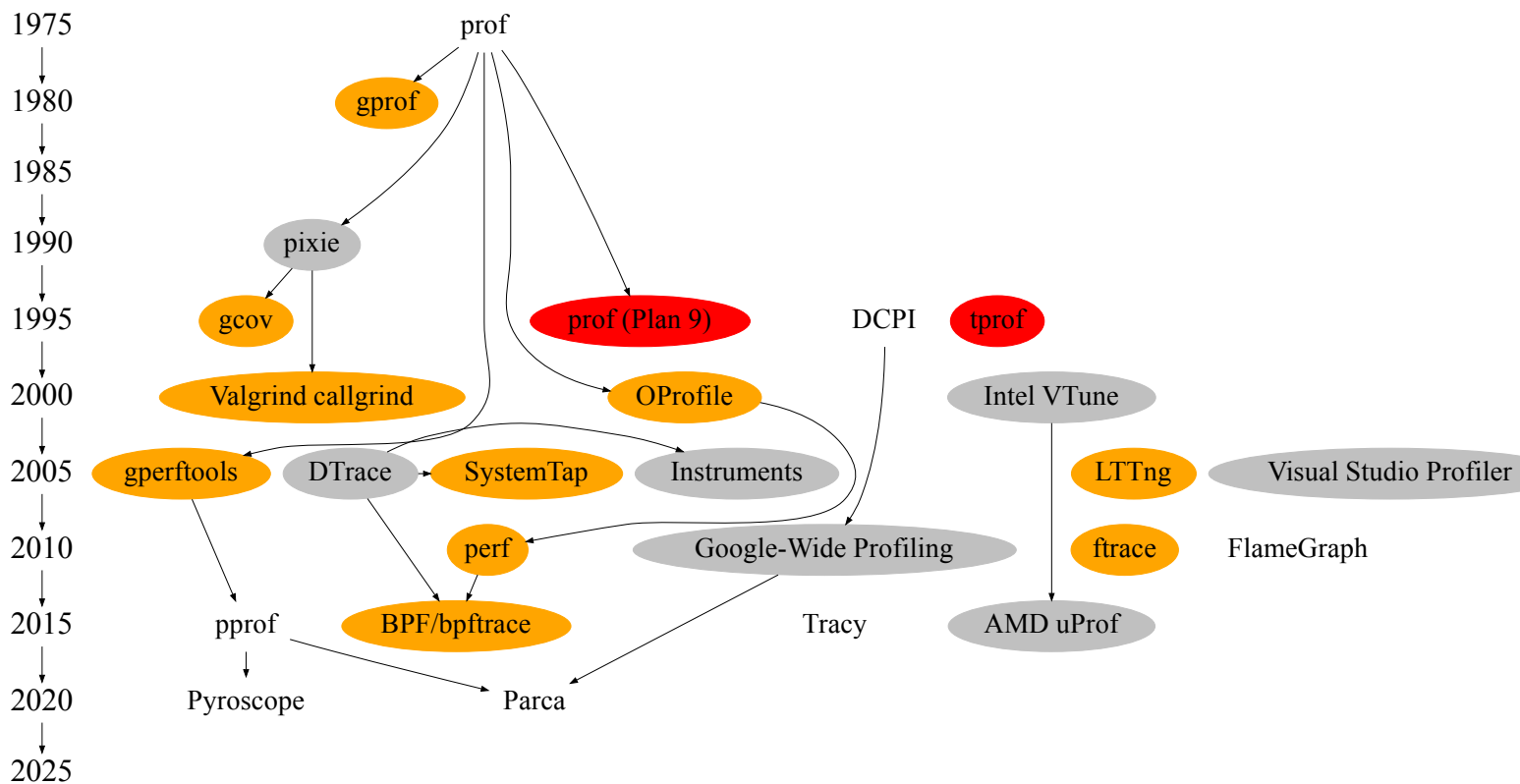


Figure 1.1: Profilers timeline

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

Chapter 2

Overview

In this chapter, I will give a bird’s-eye view of the Plan 9 profiling tools before diving into the code in later chapters. I will first recall general profiling principles, then present the command-line interfaces of each tool, show a sample profiling session, and finally describe the software architecture and code organization.

2.1 Profiler principles

There are two fundamental approaches to profiling. The first is instrumentation: the toolchain (compiler or linker) inserts extra code at each function entry and exit to record call counts and elapsed time. This is what Plan 9’s `prof` does—the linker `5l -p` inserts calls to `_profin` and `_profout` around every function. The result is a precise call tree with exact counts, but the overhead can be significant.

The second approach is sampling: the kernel periodically interrupts the running program (typically on each clock tick) and records its current program counter. After enough samples, the histogram reveals where the program spends most of its time. This is what `tprof` does, via the `/proc/<pid>/profile` file. Sampling has low overhead but gives only statistical estimates—it cannot tell you how many times a function was called, only how often the CPU was found executing within it.

2.1.1 User time vs system time vs real time

Before diving into the profiling tools themselves, it is worth clarifying a few concepts that come up repeatedly.

When you time a program, three numbers are reported: user time, system time, and real time (also called wall-clock time). User time is the time the CPU spent executing the program’s own code. System time is the time the CPU spent in the kernel on behalf of the program (handling system calls, page faults, etc.). Real time is the elapsed time from start to finish, including time the program spent waiting (for I/O, for the scheduler to give it a turn, etc.). A program that does heavy computation will have user time close to real time. A program that does lots of I/O will have high system time. A program that sleeps or waits for input will have real time much larger than user + system.

2.1.2 Ticks

How does the kernel actually measure these times? It uses a periodic timer interrupt called the clock tick. On each tick (at a frequency called HZ, e.g., 100 or 1000 times per second), the kernel interrupts whatever is running and updates the current process’s time accounting: if the process was in user mode, its user time is incremented; if in kernel mode, its system time.

This tick-based accounting is coarse-grained: the kernel simply checks `p->insyscall` when the tick fires and increments user or system time accordingly. If a short syscall starts and finishes between two ticks, it is never seen in kernel mode and the time goes to user. Conversely, if the tick fires during a syscall, the entire tick counts

as system time. (The `Tos` structure used by `prof` provides a separate, cycle-accurate mechanism via `kcycles` and `pcycles`—but that is for per-function profiling, not for the user/sys/real counters.) Figure 2.1 illustrates this with a concrete example.

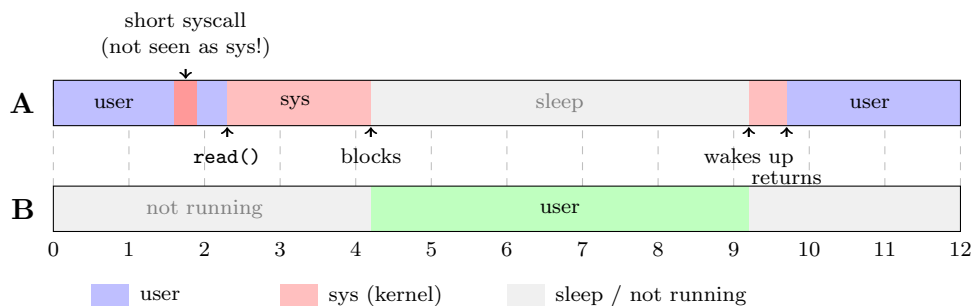


Figure 2.1: Tick-based time accounting with two interleaved processes. Ticks are numbered 0–12 along the bottom. The short syscall between ticks 1 and 2 is invisible to tick accounting (counted as user). The `read()` blocks shortly after tick 4 and A wakes up shortly after tick 9. The syscall returns at 9.7, but tick 10 is counted as user because `insyscall` is already false when it fires. After tick 12: A has `user=5`, `sys=2`, `real=12`; B has `user=5`, `sys=0`, `real=8`.

Note that the timer interrupt handler itself is kernel code, but it executes so quickly (a few microseconds) compared to a full tick interval (10 milliseconds at `HZ=100`) that its cost is negligible and not separately accounted for.

2.1.3 Timer interrupt and preemptive scheduling

The tick interrupt is also what makes preemptive scheduling possible: it gives the kernel a chance to check whether the current process has used its time slice and should yield the CPU to another process.

Profilers exploit this interrupt in two ways. First, the tick is when per-process time counters are updated (which timing tools eventually read). Second, a separate timer at a different frequency can sample the program counter for sampling-based profiling. Using a frequency that is not a multiple of `HZ` avoids aliasing—without this, the profiler might systematically miss code that runs in sync with the clock. The idea behind program counter sampling is simple: when the timer fires, the kernel looks at the interrupted instruction’s address (the PC) and increments a counter for that address range. After many samples, functions where the program spends the most time will have the highest counts.

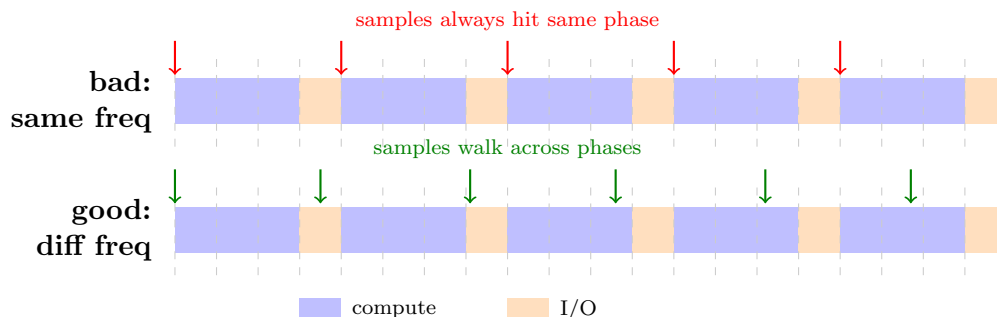


Figure 2.2: The aliasing problem. Top: when the profiling timer fires at the same frequency as the program’s repeating pattern, every sample hits the same phase (here, always “compute”), missing the I/O portion entirely. Bottom: using a different frequency (e.g., 113 instead of 100), samples gradually walk across all phases, giving a representative picture.

2.1.4 Instrumentation profilers

An instrumentation profiler modifies the program so that extra code runs at each function entry and exit. This instrumentation can collect several kinds of data:

- *Call counts*: how many times each function was called. This is the simplest form of instrumentation.
- *Per-function time*: how much time was spent inside each function, including or excluding its callees.
- *Call graph*: which function called which, and how much time was spent along each call arc.

The main advantage of instrumentation is precision: you get exact counts and a complete call tree. The main disadvantage is overhead—every function call now executes extra instructions, which can slow the program down and distort the very timings you are trying to measure. Moreover, the instrumentation needs extra data structures to store the counts and times—typically a buffer of profiling records allocated at program startup.

2.1.5 Sampling profilers

A sampling profiler takes a different approach: it periodically interrupts the program and records its current program counter (PC). Over thousands of samples, the histogram of PCs reveals which functions consume the most CPU time. In principle, the kernel could also walk the stack at each sample to record not just the current PC but also the chain of callers. Some modern profilers (like Linux's `perf`) do this, but it requires the kernel to understand the stack frame layout, which is architecture-specific and can be fragile. Plan 9's `tprof` keeps things simple and only records the top-level PC.

Sampling has two key advantages: it requires no recompilation or relinking, and it has very low overhead (just one interrupt every few milliseconds). The disadvantage is that you only get statistical estimates—a function that runs for a very short time may never be sampled, and you cannot recover call counts or call graphs from sampling data alone. (The binary must still contain a symbol table so that the profiler can map PC values back to function names, but no relinking or recompilation is needed.)

2.1.6 Tracing and monitoring profilers

The Plan 9 profiling tools also include a third category: tracing and monitoring. `trace` shows real-time scheduler events (process switches, deadlines, releases) in a graphical timeline, while `stats` displays rolling graphs of system-wide counters (syscalls/sec, context switches, memory usage, etc.). These tools do not profile a single program but rather observe the whole system. These tools are especially useful when debugging performance problems that involve multiple processes or the kernel itself—for instance, understanding why a system feels sluggish, or spotting a process that consumes too many context switches or syscalls.

2.2 time command-line interface

```
<main() (time.c) usage if no args 10a>≡ (22)
if(argc <= 1){
    fprintf(2, "usage: time command\n");
    exits("usage");
}
```

2.3 prof and tprof command-line interfaces

```
<main() (prof.c) print usage and exit 10b>≡ (26d)
fprintf(STDERR, "usage: prof [-dr] [8.out] [prof.out]\n");
exits("usage");
```

prof reads a profile data file (**prof.<pid>**) produced by an instrumented program and correlates it with the program's symbol table. For each function, it prints the percentage of time spent there, the absolute time, and the call count. The **-d** flag prints the dynamic call graph instead, showing the tree of calls indented by depth. The profiling runtime writes its data to **prof.<pid>** (not **prof.out**), so in practice you pass the filename as a second argument: **prof 5.out prof.123**.

```
<main() (tprof.c) sanity check argc and print usage 11a>≡ (36)
if(argc != 2 && argc != 3)
    error(false, "usage: tprof pid [binary]");
```

tprof works differently: it reads **/proc/<pid>/profile**, which contains interrupt-time samples of the program counter collected by the kernel. Since sampling cannot track calls, **tprof** has no **-d** (call graph) option—it only reports time per function.

To enable sampling, you first write “**profile**” to **/proc/<pid>/ctl**, let the program run, then run **tprof <pid>** while the process is still alive.

```
<main() (kprof.c) sanity check argc and print usage 11b>≡ (41)
if(argc != 3)
    error(0, "usage: kprof text data");
```

kprof is a variant of **tprof** that profiles the kernel itself. It reads the data accumulated by the **kprof(3)** device (**/dev/kpdata**), which records one count per clock tick per 8-byte range of kernel text.

2.4 trace and stats graphical user interfaces

trace and **stats** are graphical programs that display system activity in real time. **trace** reads events from **/proc/trace** and draws a timeline for each traced process: colored blocks show when a process is running, and arrows mark scheduling events (releases, deadlines, yields, overruns). The user can zoom in and out with **+/-** and pause with **p**.

stats reads counters from kernel pseudo-files (**#c/sysstat**, **#c/swap**, **/net/ether0/0/stats**) and plots rolling graphs updated once per second. It can display dozens of metrics: system load, memory usage, context switches, syscalls, interrupts, TLB misses, network packets, and more. It can even monitor multiple machines simultaneously, drawing their graphs in adjacent columns. **stats** is similar in spirit to tools like GNOME System Monitor on Linux or Activity Monitor on macOS, but it runs in a terminal-sized window and focuses on kernel-level counters rather than per-process resource usage.

The lower-case flags select which metrics to display. The most commonly used are **-l** for system load (the default), **-m** for memory usage, **-c** for context switches per second, **-s** for syscalls per second, **-i** for interrupts per second, and **-e** for network packets. Upper-case flags control the display: **-S** sets a scale factor, **-L** switches to logarithmic axes, and **-Y** adds value markers along the y axis.

```
<function usage (misc/stats.c) 11c>≡ (92)
void
usage(void)
{
    fprintf(2, "usage: stats [-0] [-S scale] [-LY] [-%s] [machine...]\n", argchars);
    exits("usage");
}
```

2.5 iostats filesystem interface

iostats takes yet another approach: it is a user-level file server that interposes itself between a program and the regular file server, intercepting all 9P messages. When the program exits, **iostats** prints a report showing how much data was read and written, the number and latency of each 9P message type, and a per-file I/O summary.

This is a classic Plan 9 design: instead of adding tracing hooks inside the kernel, you insert a proxy file server in the namespace. For example, `iostats ls` runs `ls` through the proxy and reports how many `read`, `stat`, and `open` messages that simple command generates.

2.6 Profiling hello

To get a feel for the tools, consider profiling the simplest possible program, `hello.c`:

```
<hello.c 12a>≡
#include <u.h>
#include <libc.h>

void main() {
    print("hello world\n");
    exits(nil);
}
```

For a more interesting example, consider a program that calls `foo` in a loop and `bar` once, where `foo` itself calls `bar` as in `funcs.c`:

```
<funcs.c 12b>≡
#include <u.h>
#include <libc.h>

void bar(void) { /* some work */ }
void foo(void) { bar(); /* more work */ }
void main(void) {
    int i;
    for(i = 0; i < 100; i++)
        foo();
    bar();
    exits(nil);
}
```

With time:

```
% time hello
hello world
0.00u 0.00s 0.01r    hello
```

The three numbers are user time, system time, and real (wall-clock) time, all in seconds.

With `prof`, you first link with `-p` to enable instrumentation, run the program, then display the results:

```
% 5l -p -o hello hello.5
% ./hello
hello world
% prof hello hello.42
%      Time    Calls  Name
80.0   0.004      1  main
20.0   0.001      1  libc_print
0.0    0.000      1  memset
0.0    0.000      1  getpid
0.0    0.000      1  read
...
```

For `funcs` we get instead:

```
% 5l -p -o funcs funcs.5
% ./funcs
% prof funcs funcs.42
%      Time    Calls  Name
100.0   0.003      1  main
   0.0   0.000      1  memset
   0.0   0.000     101  bar
   0.0   0.000     100  foo
...
```

The `-d` flag shows the dynamic call graph instead:

```
% prof -d funcs funcs.42
main:3
. foo:0/100
.   bar:0/100
. bar:0
. getpid:0
.   memset:0
...
```

This reveals that `bar` was called from two sites: 100 times from `foo` and once directly from `main`.

With `tprof`, you start a longer-running program, enable profiling, and then read the sampling data:

```
% longrun &
% echo profile > /proc/$apid/ctl
% # ... let it run ...
% tprof $apid
%      Time  Name
45.0    4.50  compute
30.0    3.00  process
15.0    1.50  sort
10.0    1.00  main
```

(In `rc`, `$apid` holds the pid of the last process started with `&`.)

With `iostats`, you see the 9P traffic:

```
% iostats hello
hello world
read      32 bytes, 1 ops      ...
write     12 bytes, 1 ops      ...
protocol  48 bytes
rpc        4
```

2.7 Code organization

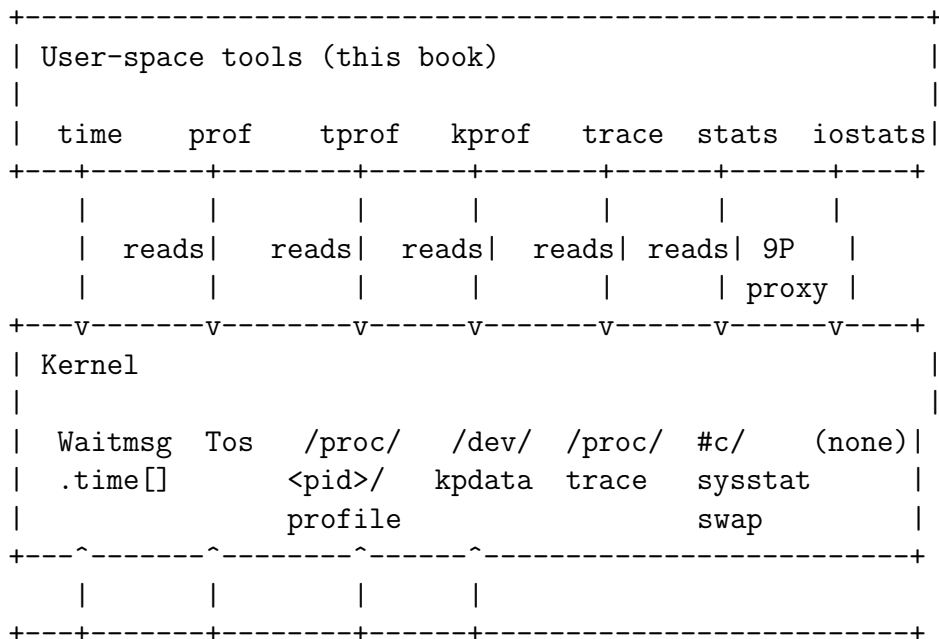
Table 2.1 presents the source files of the profiling tools, together with the main entities (e.g., structures, functions) each file defines and the corresponding chapter in which its code is discussed.

Function	Ch.	File	Entities	LOC
timing a command	5	misc/time.c	main()X add()X	111
profiling data structures	6	misc/prof.c	Data ^{25a} Pc ^{26c} Acc ^{26a}	414
profiling display	6	misc/prof.c	main()X syms() ^{28b} graph() ^{33a} plot() ^{29e}	
sampling profiler	7	misc/tprof.c	main()X COUNTER	159
kernel profiler	8	misc/kprof.c	main()X COUNTER	154
trace data structures	9	misc/trace.c	TEvent ^{45a} Task ^{45b}	841
trace GUI	9	misc/trace.c	threadmain() ^{46f} doevent() ^{52b} drawtrace() ⁵³	
stats data structures	10	misc/stats.c	Graph ^{58b} Machine ^{59a}	1622
stats GUI	10	misc/stats.c	main()X mkcol() ^{47a}	
iostats data structures	11	iostats/statfs.h	Fid Stats Rpc	174
iostats file server	11	iostats/iostats.c	main()	638
9P message handling	11	iostats/statsrv.c	rread() rwrite()	726
iostats globals	11	iostats/globals.c		45
Total				4884

Table 2.1: Chapters and source files of the profiling tools.

2.8 Software Architecture

The profiling infrastructure spans three layers of the system. At the bottom, the *kernel* provides the raw data: the `Waitmsg.time` array (user/sys/real times for `time`), the `/proc/<pid>/profile` file (PC sampling histogram for `tprof`), `/proc/trace` (scheduler events for `trace`), and `/dev/kpdata` (kernel PC histogram for `kprof`). In the middle, the *linker* (51 -p) instruments programs by inserting calls to `_profin/_profout` at function entry and exit, and the runtime library (`lib_core/libc/profile.c`) manages the profiling buffer and writes `prof.<pid>` on exit. At the top, the *user-space tools* presented in this book read this data, correlate it with symbol tables, and present it to the programmer.



```

| Linker (5l -p)                                     |
|                                                     |
| _profin/_profout    profclock    devkprof          |
| instrumentation    sampling    kernel sampling    |
+-----+

```

2.9 Book structure

The rest of the book is organized as follows. Chapters 3 and 4 describe the infrastructure that the profiling tools depend on: how the kernel exposes timing data and PC histograms, and how the linker instruments function calls. Then each tool gets its own chapter: `time` (Chapter 5, the simplest tool), `prof` (Chapter 6, instrumentation-based profiling), `tprof` (Chapter 7, sampling-based profiling), `kprof` (Chapter 8, kernel profiling), `trace` (Chapter 9, real-time scheduler visualization), `stats` (Chapter 10, system-wide monitoring), and `iostats` (Chapter 11, 9P tracing file server). Chapter 12 concludes.

Chapter 3

Kernel Support

In this chapter, I will summarize the kernel infrastructure that the profiling tools rely on. The full details can be found in *KERNEL* book [Pad14]; here I present just enough to understand the code in later chapters.

3.1 Ticks, HZ, and times

In Plan 9, the clock interrupt fires at a frequency HZ (typically 100 on ARM). On each tick, the kernel calls `accounttime()`, which checks the `insyscall` flag of the current process to decide whether to increment `p->time[TUser]` or `p->time[TSys]`. Real time is tracked differently: `p->time[TReal]` stores the global tick count at process creation, so elapsed real time is computed as `CPUS(0)->ticks - p->time[TReal]`.

Consider two processes A and B interleaved on a single CPU. Each tick, the kernel attributes time to whichever process happens to be running:

```
tick:  1    2    3    4    5    6    7    8    9   10
CPU:   [ A-user ] [ A-sys ] [ B-user ] [ A-user ] [ B  ]
        +u          +u +s +s   +u +u   +u +u   +u
```

After tick 10:

```
A: user=4  sys=2  real=10 (created at tick 0)
B: user=3  sys=0  real=5  (created at tick 5)
```

Process A's `user+sys` (6) is less than its real time (10) because B ran in between. Process B's real time starts from when it was created, not from tick 0. One Plan 9-specific detail not shown in Figure 2.1 is the accumulation of children's times. When a child exits, its user and system times are added to the parent's `TCUser` and `TCSys` counters. This is how `/bin/time` can report the total CPU time consumed by a command and all its descendants.

3.2 Wait message

The simplest form of profiling—timing a command—requires almost no special kernel support. Each process maintains five time counters (user, system, real, cumulative child user, cumulative child system), incremented by `accounttime()` on every clock tick. When a process exits, `pthread_exit()` packages three values (user, system, real) into the `Waitmsg` structure that the parent receives via `wait()`:

```
struct Waitmsg {
    ...
    ulong time[3]; /* user, sys, real (ms) */
    ...
};
```

The `/bin/time` command (Chapter 5) simply forks, execs the target program, calls `wait()`, and prints `w->time[0]`, `w->time[1]`, `w->time[2]`. The kernel does all the accounting work.

3.3 `/proc/<pid>/status` and `/dev/cputime`

While `Waitmsg` only gives timing for *completed* processes, these two files expose the same five counters for *running* processes. `/dev/cputime` returns the counters of the calling process (user, sys, real, child user, child sys, all in milliseconds). `/proc/<pid>/status` returns those of any process, which is useful for monitoring long-running programs without waiting for them to exit.

3.4 `/proc/<pid>/profile`

Tick-based profiling (`tprof`) uses a periodic timer that fires at a frequency deliberately not aligned with HZ (to avoid aliasing with other periodic events). On each tick, `profclock()` checks whether the current process is running in user mode and, if so, increments a counter in the `profile` array attached to the text segment.

The `profile` array is an array of `ulong` (4-byte unsigned integers), one entry per 8-byte range of the text segment:

```
profile[]:  ulong (4 bytes each)
+-----+-----+-----+-----+-----+
| total ms | bucket 0 | bucket 1 | bucket 2 | ... |
+-----+-----+-----+-----+-----+
index: 0           1           2           3

text segment:
+-----+-----+-----+-----+-----+-----+
| 8 bytes| 8 bytes| 8 bytes| 8 bytes| 8 bytes| ... |
+-----+-----+-----+-----+-----+-----+
base      base+8  base+16  base+24  base+32
      ^         ^         ^         ^
      |         |         |         |
    bucket 0 bucket 1 bucket 2 bucket 3
```

The array covers the entire text segment with a resolution of 8 bytes per bucket (`LRESPROF = 3`, so `pc >> 3` gives the bucket index). The first element `profile[0]` holds the total execution time in milliseconds; subsequent elements hold per-bucket times. The profile data is attached to the *segment*, not the process, so multi-threaded programs sharing the same text accumulate a single combined profile. To activate profiling, the user writes “profile” to `/proc/<pid>/ctl`, which allocates the `profile` array. The `/bin/tprof` tool then reads `/proc/<pid>/profile` and correlates the buckets with symbol information from the binary.

3.5 `/proc/trace`

The `/proc/trace` file provides a stream of scheduler events for processes that have tracing enabled. Each event is a `Traceevent` structure containing a process ID, an event type (`SReady`, `SRun`, `SDead`, `SSleep`, or `SUser`), and a timestamp. The events are stored in a kernel-side circular buffer. Hooks in `ready()`, `runproc()`, `sleep()`, and `pexit()` generate events whenever a process changes state. To enable tracing for a process, the user writes “trace” to `/proc/<pid>/ctl`. The `/bin/trace` tool reads the event stream and draws the graphical timeline. The event types are defined in `/sys/include/trace.h`, which is shared between the kernel and `/bin/trace`.

3.6 TOS

Function-level profiling (**prof**) needs to measure the time spent in each function. The naive approach would be to call a time function (like **nsec()** or **times()**) at each function entry and exit to record timestamps. But these functions are system calls—they trap into the kernel, which is orders of magnitude slower than a regular function call. A program with thousands of small functions would spend more time measuring than computing.

The kernel solves this problem by mapping the **Tos** (Top of Stack) structure at the top of every user stack, giving user code direct read access to timing information without entering the kernel:

```
struct Tos {
    struct { ... } prof; /* profiling linked list */
    uulong cyclefreq;    /* cycle counter frequency */
    vlong kcycles;       /* cycles in kernel */
    vlong pcycles;       /* cycles in process */
    ulong clock;         /* millisecond clock */
    ulong pid;
    ...
};
```

The **Tos** structure sits at the very top of the user stack, just below **USTKTOP**. The kernel can write to it (it is mapped in the kernel's address space too), and user code can read it directly without a syscall:

```
USTKTOP
+-----+
| Tos                | <-- tos = USTKTOP - sizeof(Tos)
|   prof.pp, prof.next ... | \
|   cyclefreq         | | read by _profin/_profout
|   kcycles <-- updated | | in user space
|   pcycles   on every  | |
|   clock      kernel   | /
|   pid        entry/exit |
+-----+
| exec arguments      |
+-----+
|
| user stack frames   |
| (grows downward)   |
|                     |
|                     |
|                     |
|                     |
+-----+
|                     |
|                     |
+-----+
```

The kernel updates **tos->kcycles** and **tos->pcycles** on every kernel entry and exit (in **syscall()**, **trap()**, and **arch__kexit()**), and **tos->clock** on every profiling tick. The profiling runtime library (**lib_core/libc/profil**) reads these fields to compute per-function times entirely in user space. The **prof** field in **Tos** holds the linked list of profiling records that the linker's instrumentation code (**_profin/_profout**) maintains. When the program exits, the runtime writes this data to **prof.<pid>**.

Chapter 4

Linker Support

In this chapter, I will summarize how the linker 5l instruments programs for profiling. The full details can be found in LINKER book [Pad15]; here I present just enough to understand how `prof` gets its data.

Surprisingly, in Plan 9 profiling is enabled by a flag of the *linker* (5l -p), not the compiler. This is possible because the linker operates on the instruction stream and can insert extra instructions after `TEXT` (function entry) and before `RET` (function return). The linker supports two profiling strategies, selected by 5l -p (the default) or 5l -p -1.

4.1 5l -p

The default strategy (5l -p) measures the *time spent in each function*. The linker inserts a `BL _profin` call right after every `TEXT` instruction and a `BL _profout` call right before every `RET`:

```
TEXT foo(SB), $0      TEXT foo(SB), $0
                        ->  BL _profin(SB)
...
                        ...
                        BL _profout(SB)
RET                  ->  RET
```

The `_profin()` and `_profout()` functions are implemented in `lib_core/libc/profile.c`. They use the `Tos` structure (described in the previous chapter) to read the current time without making a system call, and maintain a linked list of profiling records that track the call tree.

For example, if `main` calls `foo` which calls `bar`, the profiling records form a tree mirroring the call stack:

```
tos->prof.pp --> [main: 12ms, 1 call]
                  |
                  +---> [foo: 8ms, 3 calls]
                        |
                        +---> [bar: 2ms, 5 calls]
```

Each `_profin` pushes a new record (or increments an existing one), and each `_profout` pops back to the caller, accumulating the elapsed time.

The -p flag also changes the program's entry point from `_main` to `_mainp`, which calls `_profmain()` to initialize the profiling data structures before entering `main()`. When the program exits, the runtime writes the accumulated data to `prof.<pid>`, which `/bin/prof` then reads.

4.2 5l -p1

The simpler strategy (5l -p -1) only counts *how many times each function is called*, without measuring time. The linker allocates a global array `__mcount` where each function gets two 4-byte entries: one for the function's address and one for the call count. After each `TEXT` instruction, the linker inserts three instructions that load the counter, increment it, and store it back:

```
TEXT foo(SB), $0      TEXT foo(SB), $0
                        ->    MOVW __mcount+8(SB), R11
                        ADD    $1, R11
...                   MOVW R11, __mcount+8(SB)
                        ...
RET                   ->    RET
```

This strategy has lower overhead than 5l -p (no function call, just a load-increment-store) but provides less information: you learn which functions are hot by call count, but not how much time each call takes.

For a program with functions `main`, `foo`, and `bar`, the `__mcount` array in memory looks like:

```
__mcount:
offset  contents
+0      5          (total size in words: 1 + 2*nfuncs)
+4      &main      (address of main)
+8      0          (call count for main, incremented at runtime)
+12     &foo       (address of foo)
+16     0          (call count for foo)
+20     &bar       (address of bar)
+24     0          (call count for bar)
```

4.3 The symbol table

Both `prof` and `tprof` ultimately need to map a program counter (PC) value to a function name. This is done by reading the symbol table embedded in the executable. The linker produces this table: for each function, it records the function's name and the starting address of its code. Given a PC, the profiler finds the symbol whose address is closest below the PC—that is the function the PC belongs to. In Plan 9, the tool library `libmach` provides functions to read symbol tables from Plan 9 executables. Both `prof` and `tprof` use these to correlate addresses (from the profiling data or the sampling histogram) with human-readable function names. Without the symbol table, profiling data would just be a list of raw addresses.

A Plan 9 executable on disk has the following layout (see also LINKER book [Pad15]):

```
+-----+
| header          | magic, sizes, entry point
+-----+
| text segment    | machine code
+-----+
| data segment    | initialized globals
+-----+
| symbol table     | name, type, address for each symbol
+-----+
| PC/line table   | maps PC ranges to source lines
+-----+
```

symbol table entry:

```
+-----+-----+-----+
| value | type | name |
| (addr) |      | (string) |
+-----+-----+-----+
```

The profiler reads the symbol table, sorts the entries by address, and for each PC in the profiling data, performs a binary search to find the enclosing function.

Chapter 5

Timing a Command: `/bin/time`

`time` is the simplest profiling tool—barely 60 lines of C. As explained in Chapter 3, the kernel already delivers timing data in `Waitmsg`, so all `time` has to do is fork, exec the target command, call `wait()`, and print the three numbers.

5.1 `main()`

```
<function main(time.c) 22>≡ (68a)
void
main(int argc, char *argv[])
{
    Waitmsg *w;
    long l;
    <main()(time.c) other locals 23d>

    <main()(time.c) usage if no args 10a>

    switch(fork()){
    case 0:
        // in child
        exec(argv[1], &argv[1]);

        <main()(time.c) in child after exec 23c>
        error(argv[1]);
    case -1:
        error("fork");
    }
    // else, in parent

    notify(notifyf);

    loop:
    w = wait();
    <main()(time.c) if w == nil 24a>
    l = w->time[0];
    add("%ld.%2ldu", l/1000, (l%1000)/10);
    l = w->time[1];
    add("%ld.%2lds", l/1000, (l%1000)/10);
    l = w->time[2];
    add("%ld.%2ldr", l/1000, (l%1000)/10);
    add("\t");

    <main()(time.c) display also argv 24d>
    <main()(time.c) display also wait message and status 24e>
```

```

    fprintf(STDERR, "%s\n", output);
    exits(w->msg);
}

```

The three time values in `Waitmsg` are in milliseconds. The formatting divides by 1000 for the integer part and extracts hundredths from the remainder, appending `u` (user), `s` (system), or `r` (real). For example, if `w->time[0]` is 15050 (milliseconds), the output is 15.05u ($15050/1000 = 15$, $(15050\%1000)/10 = 5$).

```

⟨global output(time.c) 23a⟩≡ (68a)
char output[4096];

```

```

⟨function add(time.c) 23b⟩≡ (68a)
void
add(char *a, ...)
{
    static bool beenhere=false;
    va_list arg;

    if(beenhere)
        strcat(output, " ");
    va_start(arg, a);
    vseprint(output+strlen(output), output+sizeof(output), a, arg);
    va_end(arg);
    beenhere = true;
}

```

Unlike UNIX's `execvp`, Plan 9's `exec()` does not search a path—it requires an exact file name. In Plan 9, path searching is the shell's job (using the `path` variable in `rc`). Here `time` does a minimal lookup by trying `/bin/` if the bare name fails. This is sufficient in practice because Plan 9 uses union mounts to bind all binary directories into `/bin/`, unlike UNIX which scatters binaries across `/usr/bin`, `/usr/local/bin`, etc.

```

⟨main() (time.c) in child after exec 23c⟩≡ (22)
if(argv[1][0] != '/' && strncmp(argv[1], "./", 2) &&
    strncmp(argv[1], "../", 3)){
    sprintf(output, "/bin/%s", argv[1]);
    exec(output, &argv[1]);
}

```

5.2 Error management

```

⟨main() (time.c) other locals 23d⟩≡ (22) 24c▷
char err[ERRMAX];

```

```

⟨function error(time.c) 23e⟩≡ (68a)
static void
error(char *s)
{
    fprintf(STDERR, "time: %s: %r\n", s);
    exits(s);
}

```

If the user presses the interrupt key (e.g., Delete) while the child is running, `wait()` returns `nil` with the error string “interrupted”. In that case, `time` simply retries `wait()`—the interrupt was meant for the child, not

for `time` itself. The `notifyf` handler registered via `notify()` ensures that interrupts are caught (`NCONT`) rather than killing `time`.

```
<main() (time.c) if w == nil 24a>≡ (22)
    if(w == nil){
        errstr(err, sizeof err);
        if(strcmp(err, "interrupted") == 0)
            goto loop;
        error("wait");
    }
```

```
<function notifyf(time.c) 24b>≡ (68a)
    void
    notifyf(void *a, char *s)
    {
        USED(a);
        if(strcmp(s, "interrupt") == 0)
            noted(NCONT);
        noted(NDFLT);
    }
```

5.3 Extra display

Besides the timing data, `time` also prints the command line (truncated to 4 arguments) and, if the child exited with an error, the exit status string.

See for instance:

```
% time ls /xxx
ls: /xxx: '/xxx' file does not exit
0.00u 0.01s 0.02r      ls /xxx # status= errors
%
```

```
<main() (time.c) other locals 24c>+≡ (22) <23d
    int i;
    char *p;
```

```
<main() (time.c) display also argv 24d>≡ (22)
    for(i=1; i<argc; i++){
        add("%s", argv[i], 0);
        if(i>4){
            add("...");
            break;
        }
    }
```

The exit status in Plan 9 is a string (not a number). By convention it has the form “`program: message`”, so `time` strips the program name prefix and shows just the message part after the colon.

```
<main() (time.c) display also wait message and status 24e>≡ (22)
    if(w->msg[0]){
        p = utfrune(w->msg, ':');
        if(p && p[1])
            p++;
        else
            p = w->msg;
        add(" # status=%s", p);
    }
```

Chapter 6

Instrumentation-based Profiler: `/bin/prof`

`prof` reads the profiling data written by the instrumented program (the `prof.<pid>` file) and the symbol table from the executable, then presents the results either as a flat profile (the default) or as an indented call graph (`-d` flag). Recall the example from Section 2.6: the flat profile shows per-function time and call counts, while `-d` shows which function called which.

6.1 Data structures

The profiling data file contains an array of `Data`^{25a} records written by the runtime library (`lib_core/libc/profile.c`). Each record represents a function call in the dynamic call tree:

```
<struct Data 25a>≡ (68c)
struct Data
{
    ushort down;
    ushort right;

    ulong pc;
    ulong count;
    ulong time;
};
```

The `down` and `right` fields encode a tree using array indices (not pointers): `down` points to the first callee, `right` to the next sibling at the same call depth. The sentinel value `0xFFFF` means “no child” or “no sibling.” The `pc` field identifies the function, `count` is the number of times it was called at this point in the tree, and `time` is the cumulative time in milliseconds (including callees).

```
<global data 25b>≡ (68c)
// ref_own<array<Data> (len = ndata)
Data* data;
```

```
<global ndata 25c>≡ (68c)
// len(data)
long ndata;
```

For the example program from Section 2.6 where `main` calls `foo` 100 times and `bar` once, and `foo` calls `bar`, the `data` array encodes the following tree:

<code>data[0]:</code>	<code>pc=main</code>	<code>count=1</code>	<code>time=100</code>	<code>down=1</code>	<code>right=FFFF</code>
<code>data[1]:</code>	<code>pc=foo</code>	<code>count=100</code>	<code>time=80</code>	<code>down=3</code>	<code>right=2</code>
<code>data[2]:</code>	<code>pc=bar</code>	<code>count=1</code>	<code>time=10</code>	<code>down=FFFF</code>	<code>right=FFFF</code>
<code>data[3]:</code>	<code>pc=bar</code>	<code>count=100</code>	<code>time=20</code>	<code>down=FFFF</code>	<code>right=FFFF</code>

```

Tree:      main (100ms, 1 call)
           |
           +--down--> foo (80ms, 100 calls)
           |         |
           |         +---down--> bar (20ms, 100 calls)
           |
           +---right--> bar (10ms, 1 call)

```

At runtime, `_profin()` and `_profout()` maintain a linked list of `Plink` records (stored in `Tos.prof`) that track the current call chain. When the program exits, `_profmain()` calls `_profdump()` which serializes this linked list into the flat `Data` array written to `prof.<pid>`.

The `Acc`^{26a} structure is used to accumulate a flat per-function summary from the tree: for each function, it records the function's name, address, total *self* time (excluding callees), and call count.

```

<struct Acc 26a>≡ (68c)
struct Acc
{
    char *name;
    ulong pc;
    ulong ms;
    ulong calls;
};

```

```

<global acc 26b>≡ (68c)
Acc* acc;

```

The `Pc`^{26c} structure is only used by `graph()`^{33a} to detect recursion when printing the call graph with `-d`. It forms a linked list of PCs representing the current call chain during the tree walk.

```

<struct Pc 26c>≡ (68c)
struct Pc
{
    Pc *next;
    ulong pc;
};

```

6.2 main()

The main function has three phases: load the symbol table from the executable via `syms()`^{28b}, load the profiling data from `prof.out` via `datas()`^{28e}, then display the results via either `plot()`^{29e} (flat profile) or `graph()`^{33a} (call graph with `-d`).

```

<function main (misc/prof.c) 26d>≡ (68c)
void
main(int argc, char *argv[])
{
    char *s;

    <main() (prof.c) adjust tabstop 34b>

    ARGBEGIN{
    <main() (prof.c) command-line parsing 27c>
    default:
        <main() (prof.c) print usage and exit 10b>
    }ARGEND

    Binit(&bout, STDOUT, OWRITE);

```

```

    if(argc > 0)
        syms(argv[0]);
    else
        syms(defaout());

    if(argc > 1)
        datas(argv[1]);
    else
        datas("prof.out");

    if(ndata){
        <main() (prof.c) if dflag 32d>
        else
            plot();
    }
    exits(nil);
}

<global bout 27a>≡ (68c)
    Biobuf bout;

<global verbose 27b>≡ (68c)
    int verbose;

<main() (prof.c) command-line parsing 27c>≡ (26d) 32c▷
    case 'v':
        verbose = 1;
        break;

    If no executable is given on the command line, prof defaults to the standard output name for the current
    architecture (e.g., 5.out for ARM, 8.out for x86), determined from the objtype environment variable.

<function defaout 27d>≡ (68c)
    char*
    defaout(void)
    {
        char *p;
        int i;

        p = getenv("objtype");
        if(p)
            for(i=0; trans[i]; i+=2)
                if(strcmp(p, trans[i]) == 0)
                    return trans[i+1];
        return trans[1];
    }

<global trans 27e>≡ (68c)
    char* trans[] =
    {
        "386", "8.out",
        "68020", "2.out",
        "alpha", "7.out",
        "amd64", "6.out",
        "arm", "5.out",
        "mips", "v.out",
        "power", "q.out",
        "sparc", "k.out",
        "spim", "0.out",
        0,0
    };

```

6.3 Symbol table loading: syms()

`syms()`^{28b} opens the executable, parses its header with `crackhdr()`, and initializes the symbol table with `syminit()`. These are `libmach` library functions. After this call, functions like `textsym()` and `findsym()` can be used to look up function names by address.

```
<global nsym 28a>≡ (68c)
    long nsym;
```

```
<function syms 28b>≡ (68c)
    void
    syms(char *cout)
    {
        Fhdr f;
        fdt fd;

        fd = open(cout, 0);
        <syms() sanity check fd 28c>
        if (!crackhdr(fd, &f)) {
            fprintf(STDERR, "can't read text file header\n");
            exits("read");
        }
        <syms() sanity check f.type 28d>
        if (syminit(fd, &f) < 0) {
            fprintf(STDERR, "syminit: %r\n");
            exits("syms");
        }
        close(fd);
    }
```

```
<syms() sanity check fd 28c>≡ (28b)
    if(fd < 0){
        perror(cout);
        exits("open");
    }
```

```
<syms() sanity check f.type 28d>≡ (28b)
    if (f.type == FNONE) {
        fprintf(STDERR, "text file not an a.out\n");
        exits("file type");
    }
```

6.4 Profiling data loading: datas()

`datas()`^{28e} reads the `prof.out` file, which is simply a flat array of `Data`^{25a} records written by the profiling runtime. The number of records is computed from the file size. Since Plan 9 uses big-endian encoding for the profile data (regardless of the host architecture), each record must be byte-swapped via `swapdata()`^{29a}.

```
<function datas 28e>≡ (68c)
    void
    datas(char *dout)
    {
        fdt fd;
        Dir *d;
        int i;

        fd = open(dout, 0);
        <datas() sanity check fd 29b>
```

```

    d = dirfstat(fd);
    <datas() sanity check d 29c>
    ndata = d->length/sizeof(data[0]);
    data = malloc(ndata*sizeof(Data));
    <datas() sanity check data 29d>
    if(read(fd, data, d->length) != d->length){
        fprintf(STDERR, "prof: can't read data file\n");
        exits("data read");
    }
    free(d);
    close(fd);
    for (i = 0; i < ndata; i++)
        swapdata(data+i);
}

<function swapdata 29a>≡ (68c)
void
swapdata(Data *dp)
{
    dp->down = beswab(dp->down);
    dp->right = beswab(dp->right);
    dp->pc = beswal(dp->pc);
    dp->count = beswal(dp->count);
    dp->time = beswal(dp->time);
}

<datas() sanity check fd 29b>≡ (28e)
if(fd < 0){
    perror(dout);
    exits("open");
}

<datas() sanity check d 29c>≡ (28e)
if(d == nil){
    perror(dout);
    exits("stat");
}

<datas() sanity check data 29d>≡ (28e)
if(data == 0){
    fprintf(STDERR, "prof: can't malloc data\n");
    exits("data malloc");
}

```

6.5 Displaying data: plot()

`plot()`^{29e} produces the flat profile. It first builds the `acc`^{26b} array by iterating over all text symbols, then calls `sum()`^{30c} to walk the call tree and accumulate per-function self time. Finally, it sorts by time (ascending, so the hottest functions come last) and prints each function with a non-zero call count.

```

<function plot 29e>≡ (68c)
void
plot(void)
{
    Symbol s;

    for (nsym = 0; textsym(&s, nsym); nsym++) {
        acc = realloc(acc, (nsym+1)*sizeof(Acc));
    }
}

```

```

    <plot() sanity check acc 30b>
    acc[nsym].name = s.name;
    acc[nsym].pc = s.value;
    acc[nsym].calls = acc[nsym].ms = 0;
}

sum(data[0].down);
qsort(acc, nsym, sizeof(Acc), acmp);

Bprint(&bout, "   %%      Time      Calls  Name\n");
if(ms == 0)
    ms = 1;

while (--nsym >= 0) {
    if(acc[nsym].calls)
        Bprint(&bout, "%4.1f %8.3f %8lud\t%s\n",
            (100.0*acc[nsym].ms)/ms,
            acc[nsym].ms/1000.0,
            acc[nsym].calls,
            acc[nsym].name);
}
}

```

For the `foo/bar` example, after `sum()` walks the tree, the `acc` array contains self times (total minus children):

```

acc[]:
name  pc      ms  calls
main  0x1000   10    1      (100 - 80 - 10 = 10)
foo   0x1040   60   100     (80 - 20 = 60)
bar   0x1080   30   101     (20 + 10 = 30)

```

After `qsort` (ascending by `ms`) and printing (descending):

```

%      Time      Calls  Name
60.0    0.060     100   foo
30.0    0.030     101   bar
10.0    0.010       1  main

```

```

<global ms 30a>≡ (68c)
    ulong ms;

```

```

<plot() sanity check acc 30b>≡ (29e)
    if(acc == 0){
        fprintf(2, "prof: malloc fail\n");
        exits("acc malloc");
    }

```

`sum()` recursively walks the call tree encoded in the `data`^{25b} array. For each node, it computes the *self time* by subtracting the children's time (`dtype`) from the node's total time. This self time is attributed to the function identified by `symind()`^{31c} (a binary search in the sorted `acc` array). The **right** links are followed to accumulate sibling times, returning the total time of the subtree.

```

<function sum 30c>≡ (68c)
    ulong
    sum(ulong i)
    {
        long j, dtype, time;
        int k;
        static int indent;

```

```

⟨sum() sanity check i 31a⟩

j = symind(data[i].pc);
time = data[i].time;
if(time < 0)
    time += data[0].time;

⟨sum() if verbose 31b⟩
dtime = 0;
if(data[i].down != 0xFFFF){
    indent++;
    dtime = sum(data[i].down);
    indent--;
}
j = symind(data[i].pc);
if (j >= 0) {
    acc[j].ms += time - dtime;
    ms += time - dtime;
    acc[j].calls += data[i].count;
}
if(data[i].right == 0xFFFF)
    return time;
return time + sum(data[i].right);
}

```

The traversal of `sum()` for the example tree proceeds as follows:

```

sum(0) -- main: time=100
|  sum(1) -- foo: time=80      (down from main)
|    |  sum(3) -- bar: time=20 (down from foo)
|    |  return 20
|    dtime=20, self=80-20=60 -> acc[foo]+=60
|    sum(2) -- bar: time=10    (right from foo)
|    dtime=0,  self=10-0=10   -> acc[bar]+=10
|    return 80+10=90
dtime=90, self=100-90=10      -> acc[main]+=10

```

```

⟨sum() sanity check i 31a⟩≡ (30c)
if(i >= ndata){
    fprintf(STDERR, "prof: index out of range %ld [max %ld]\n", i, ndata);
    return 0;
}

```

```

⟨sum() if verbose 31b⟩≡ (30c)
if (verbose){
    for(k = 0; k < indent; k++)
        print(" ");
    print("%lud: %ld/%lud", i, data[i].time, data[i].count);
    if (j >= 0)
        print(" %s\n", acc[j].name);
    else
        print(" 0x%lux\n", data[i].pc);
}

```

`symind()` maps a PC to an index in the `acc` array using binary search. It finds the entry whose address is the largest one not exceeding the PC—that is the function the PC belongs to.

```

⟨function symind 31c⟩≡ (68c)
/*
 * assume acc is ordered by increasing text address.

```



```

    */
long
symind(ulong pc)
{
    int top, bot, mid;

    bot = 0;
    top = nsym;
    for (mid = (bot+top)/2; mid < top; mid = (bot+top)/2) {
        if (pc < acc[mid].pc)
            top = mid;
        else
            if (mid != nsym-1 && pc >= acc[mid+1].pc)
                bot = mid;
            else
                return mid;
    }
    return -1;
}

⟨function acmp 32a⟩≡ (68c)
int
acmp(void *va, void *vb)
{
    Acc *a, *b;
    ulong ua, ub;

    a = va;
    b = vb;
    ua = a->ms;
    ub = b->ms;

    if(ua > ub)
        return 1;
    if(ua < ub)
        return -1;
    return 0;
}

```

6.6 Call graph, prof -d

With the `-d` flag, `prof` displays the dynamic call graph instead of a flat profile. The `graph()`^{33a} function walks the same `Data`^{25a} tree as `sum()`^{30c}, but instead of accumulating totals, it prints each node indented by call depth, showing the function name, time, and call count.

```

⟨global dflag 32b⟩≡ (68c)
int dflag;

⟨main() (prof.c) command-line parsing 32c⟩+≡ (26d) ◁27c 34e▷
case 'd':
    dflag = 1;
    break;

⟨main() (prof.c) if dflag 32d⟩≡ (26d)
if(dflag)
    graph(0, data[0].down, 0);

```

To handle recursive functions, `graph()` maintains a linked list of `Pc`^{26c} nodes representing the current call chain. Before descending into a child, it checks whether the same PC already appears in the chain. If so, it prints “...” to indicate the recursion and stops. The `-r` flag disables this compression, printing the full (potentially very long) recursive call graph.

`<function graph 33a>≡` (68c)

```
void
graph(int ind, ulong i, Pc *pc)
{
    long time, count, prgm;
    Pc lpc;

    <graph() sanity check i 33b>
    count = data[i].count;
    time = data[i].time;
    prgm = data[i].pc;
    if(time < 0)
        time += data[0].time;
    if(data[i].right != 0xFFFF)
        graph(ind, data[i].right, pc);
    indent(ind);
    if(count == 1)
        Bprint(&bout, "%s:%lud\n", name(prgm), time);
    else
        Bprint(&bout, "%s:%lud/%lud\n", name(prgm), time, count);
    if(data[i].down == 0xFFFF)
        return;
    lpc.next = pc;
    lpc.pc = prgm;

    if(!rflag){
        while(pc){
            if(pc->pc == prgm){
                indent(ind+1);
                Bprint(&bout, "... \n");
                return;
            }
            pc = pc->next;
        }
    }

    graph(ind+1, data[i].down, &lpc);
}
```

`<graph() sanity check i 33b>≡` (33a)

```
if(i >= ndata){
    fprintf(STDERR, "prof: index out of range %ld [max %ld]\n", i, ndata);
    return;
}
```

`<function name 33c>≡` (68c)

```
char*
name(ulong pc)
{
    Symbol s;
    static char buf[16];

    if (findsym(pc, CTEXT, &s))
        return(s.name);
    snprintf(buf, sizeof(buf), "%#lux", pc);
}
```

```

        return buf;
    }

⟨global tabstop 34a⟩≡ (68c)
    int tabstop = 4;

⟨main() (prof.c) adjust tabstop 34b⟩≡ (26d)
    s = getenv("tabstop");
    if(s!=nil && strtol(s,0,0)>0)
        tabstop = strtol(s,0,0);

⟨function indent 34c⟩≡ (68c)
    void
    indent(int ind)
    {
        int j;

        j = 2*ind;
        while(j >= tabstop){
            Bwrite(&bout, ".\t", 2);
            j -= tabstop;
        }
        if(j)
            Bwrite(&bout, ".          ", j);
    }

```

6.6.1 prof -d -r

```

⟨global rflag 34d⟩≡ (68c)
    int rflag;

⟨main() (prof.c) command-line parsing 34e⟩+≡ (26d) ◁32c
    case 'r':
        rflag = 1;
        break;

```

Chapter 7

Sampling-based Profiler: `/bin/tprof`

7.1 Example: `longrun.c`

Since `tprof` samples a running process, we need a program that runs long enough to collect meaningful data. Here is a simple example with two functions that do different amounts of work, interleaved with `sleep` calls to keep the program running long enough:

```
<longrun.c 35>≡
#include <u.h>
#include <libc.h>

long compute1(void) {
    long i, sum = 0;
    for(i = 0; i < 20000000; i++)
        sum += i;
    return sum;
}
long compute2(void) {
    long i, sum = 0;
    for(i = 0; i < 10000000; i++)
        sum += i;
    return sum;
}
void main(void) {
    int i;
    for(i = 0; i < 10000; i++){
        compute1();
        compute2();
        sleep(100); /* 100 ms */
    }
    exits(nil);
}
```

To profile this program with `tprof`:

```
% 8c longrun.c && 8l -o longrun longrun.8
% longrun &
% echo profile > /proc/$apid/ctl
% # ... wait a few seconds ...
% tprof $apid
total: 80
TEXT 00001000
      ms      %      sym
      60      75.0    compute1
```

```

    20    25.2    compute2
% ...
% tprof $apid
total: 3650
TEXT 00001000
    ms      %    sym
    2790    76.4    compute1
    860     23.5    compute2

```

The profile shows `compute1` taking roughly three times as much time as `compute2`—more than the expected 2:1 ratio of loop counts. The `sleep` calls between iterations may affect the ratio: `compute1` runs right after `sleep` returns (cold cache), while `compute2` benefits from the cache warmed by `compute1`. Note that `sleep` does not appear at all—because `sleep` blocks the process, the kernel does not sample its PC during that time. Only code that is actually running on the CPU gets sampled.

`tprof` is the sampling-based counterpart to `prof`. Instead of reading instrumentation data from a `prof.<pid>` file, it reads the PC histogram from `/proc/<pid>/profile`, which the kernel populates via `profclock()` (see Chapter 3). The tool must be run while the target process is still alive, since the profile data lives in the process’s text segment.

7.2 main()

Like `prof`, `main()` has three phases: read the symbol table (from `/proc/<pid>/text` or a given binary), read the profile data (from `/proc/<pid>/profile`), and display the results. The key difference from `prof` is that the profile data is a flat array of counters (one per 8-byte bucket), not a call tree.

```

<function main (misc/tprof.c) 36>≡ (72)
void
main(int argc, char *argv[])
{
    // for symbol table
    fdt fd; // /proc/<pid>/text and then /proc/<pid>/profile
    Fhdr f;
    // for profile data
    char filebuf[128], *file; // /proc/<pid>/profile
    Dir *d;
    ulong *data; // profile content
    // for output
    Biobuf outbuf;
    <main() (tprof.c) other locals 37a>

    <main() (tprof.c) sanity check argc and print usage 11a>

    /*
     * Read symbol table
     */
    <main() (tprof.c) read symbol table 37g>

    /*
     * Read timing data
     */
    <main() (tprof.c) read timing data 38a>

    <main() (tprof.c) displaying profile 38d>

    exits(nil);
}

```

```
⟨main() (tprof.c) other locals 37a⟩≡ (36)
```

```
long i, j, k, n;
char *name;
ulong tbase, sum;
long delta;
Symbol s;
struct COUNTER *cp;
```

```
⟨main() (tprof.c) sanity check fd 37b⟩≡ (38a 37g)
```

```
if(fd < 0)
    error(true, file);
```

```
⟨main() (tprof.c) sanity check f.type 37c⟩≡ (37g)
```

```
if (f.type == FNONE)
    error(false, "text file not an a.out");
```

```
⟨main() (tprof.c) sanity check d 37d⟩≡ (38a)
```

```
if(d == nil)
    error(true, "stat");
```

```
⟨main() (tprof.c) sanity check n 37e⟩≡ (38a)
```

```
if(n < 2)
    error(false, "data file too short");
```

```
⟨main() (tprof.c) sanity check data 37f⟩≡ (38a)
```

```
if(data == nil)
    error(true, "malloc");
```

7.3 Reading the symbol table from the binary

The symbol table loading is almost identical to `syms()` ^{28b} in `prof.c`. The only difference is that `tprof` can read the binary directly from `/proc/<pid>/text` (a convenient Plan 9 feature: each process exports its executable via the `proc` filesystem), so there is no need to know the executable's path on disk.

```
⟨main() (tprof.c) read symbol table 37g⟩≡ (36)
```

```
if(argc == 2){
    file = filebuf;
    snprintf(filebuf, sizeof filebuf, "/proc/%s/text", argv[1]);
}else
    file = argv[2];
```

```
fd = open(file, OREAD);
```

```
⟨main() (tprof.c) sanity check fd 37b⟩
```

```
if (!crackhdr(fd, &f))
    error(true, "read text header");
```

```
⟨main() (tprof.c) sanity check f.type 37c⟩
```

```
machbytype(f.type);
if (syminit(fd, &f) < 0)
    error(true, "syminit");
close(fd);
```

7.4 Reading the profile data from /proc/<pid>/profile

The profile data is a flat array of `ulong` values. As described in Chapter 3, `data[0]` holds the total execution time in milliseconds, and `data[j]` holds the time spent in the 8-byte range starting at `tbase + (j-2)*PCRES` (where `tbase` is the page-aligned start of the text segment). The data must be byte-swapped since the kernel writes in big-endian format.

```
<main() (tprof.c) read timing data 38a>≡ (36)
    file = sprintf("/proc/%s/profile", argv[1]);
    fd = open(file, OREAD);
    <main() (tprof.c) sanity check fd 37b>
    free(file);

    d = dirfstat(fd);
    <main() (tprof.c) sanity check d 37d>

    n = d->length/sizeof(data[0]);
    <main() (tprof.c) sanity check n 37e>
    data = malloc(d->length);
    <main() (tprof.c) sanity check data 37f>
    if(read(fd, data, d->length) < 0)
        error(true, "text read");
    close(fd);

    for(i=0; i<n; i++)
        data[i] = machdata->swal(data[i]);
```

7.5 Displaying profile

The display logic walks the symbol table and the profile array in parallel. For each function (identified by a pair of consecutive symbol addresses), it sums the profile buckets that fall within that function's address range. Functions with non-zero sample counts are collected into `COUNTER`^{42e} records, sorted by time, and printed. The `PCRES` constant (8) matches the kernel's `LRESPROF` bucket size.

```
<constant PCRES (misc/tprof.c) 38b>≡ (72)
#define PCRES 8
```

```
<struct COUNTER (misc/tprof.c) 38c>≡ (72)
struct COUNTER
{
    char *name; /* function name */
    long time; /* ticks spent there */
};
```

```
<main() (tprof.c) displaying profile 38d>≡ (36)
    delta = data[0]-data[1];
    print("total: %ld\n", data[0]);
    if(data[0] == 0)
        exits(nil);
    // else
    if (!textsym(&s, 0))
        error(0, "no text symbols");

    tbase = s.value & ~(mach->pgsize-1); /* align down to page */

    print("TEXT %.8lux\n", tbase);

    /*
```

```

    * Accumulate counts for each function
    */
cp = 0;
k = 0;
for (i = 0, j = (s.value-tbase)/PCRES+2; j < n; i++) {
    name = s.name; /* save name */
    if (!textsym(&s, i)) /* get next symbol */
        break;
    sum = 0;
    while (j < n && j*PCRES < s.value-tbase)
        sum += data[j++];
    if (sum) {
        cp = realloc(cp, (k+1)*sizeof(struct COUNTER));
        if (cp == 0)
            error(1, "realloc");
        cp[k].name = name;
        cp[k].time = sum;
        k++;
    }
}
if (!k)
    error(0, "no counts");
cp[k].time = 0; /* "etext" can take no time */

/*
 * Sort by time and print
 */
qsort(cp, k, sizeof(struct COUNTER), compar);
Binit(&outbuf, 1, OWRITE);
Bprint(&outbuf, "      ms      %%   sym\n");
while(--k>=0)
    Bprint(&outbuf, "%6ld\t%3lld.%1ld\t%s\n",
          cp[k].time,
          100LL*cp[k].time/delta,
          (1000LL*cp[k].time/delta)%10,
          cp[k].name);

```

<function compar (misc/tprof.c) 39a>≡ (72)

```

int
compar(void *va, void *vb)
{
    struct COUNTER *a, *b;

    a = va;
    b = vb;
    if(a->time < b->time)
        return -1;
    if(a->time == b->time)
        return 0;
    return 1;
}

```

7.6 Error management

<function error (misc/tprof.c) 39b>≡ (72)

```

static void
error(bool perr, char *s)
{

```



```
fprint(STDERR, "tprof: %s", s);
if(perr){
    fprint(STDERR, ": ");
    perror(0);
}else
    fprint(STDERR, "\n");
exits(s);
}
```

Chapter 8

Profiling the Kernel: `/bin/kprof`

`kprof` is essentially the same program as `tprof`, but for the kernel instead of a user process. It reads the profile data from `/dev/kpdata` (the `kprof(3)` device) and the symbol table from the kernel binary (e.g., `/386/9pcdisk` or `/arm/9`). The usage is:

```
% bind -a '#K' /dev
% echo start > /dev/kpctl
% # ... run workload ...
% echo stop > /dev/kpctl
% kprof /arm/9 /dev/kpdata
```

The code is nearly identical to `tprof.c`. The main differences are: (1) the symbol table comes from a file on disk (the kernel binary) rather than `/proc/<pid>/text`, (2) the text base address is `mach->kbase` (the kernel's load address) rather than a page-aligned user text address, and (3) the output distinguishes time spent “in kernel text” from time spent “outside kernel text” (e.g., in user mode or idle).

```
<function main (misc/kprof.c) 41>≡ (68b)
void
main(int argc, char *argv[])
{
    fdt fd;
    Fhdr f;
    Dir *d;
    Biobuf outbuf;
    ulong *data;
    <main() (kprof.c) other locals 42a>

    <main() (kprof.c) sanity check argc and print usage 11b>

    /*
     * Read symbol table
     */
    <main() (kprof.c) read symbol table 42b>

    /*
     * Read timing data
     */
    <main() (kprof.c) read profile data 42c>

    <main() (kprof.c) display profile data 42f>

    exits(0);
}
```

<main() (kprof.c) other locals 42a>≡ (41)

```
long i, j, k, n;
char *name;
vlong tbase;
ulong sum;
long delta;
Symbol s;
struct COUNTER *cp;
```

<main() (kprof.c) read symbol table 42b>≡ (41)

```
fd = open(argv[1], OREAD);
if(fd < 0)
    error(1, argv[1]);
if (!crackhdr(fd, &f))
    error(1, "read text header");
if (f.type == FNONE)
    error(0, "text file not an a.out");
if (syminit(fd, &f) < 0)
    error(1, "syminit");
close(fd);
```

<main() (kprof.c) read profile data 42c>≡ (41)

```
fd = open(argv[2], OREAD);
if(fd < 0)
    error(1, argv[2]);
d = dirfstat(fd);
if(d == nil)
    error(1, "stat");
n = d->length/sizeof(data[0]);
if(n < 2)
    error(0, "data file too short");
data = malloc(d->length);
if(data == 0)
    error(1, "malloc");
if(read(fd, data, d->length) < 0)
    error(1, "text read");
close(fd);
for(i=0; i<n; i++)
    data[i] = beswal(data[i]);
```

<constant PCRES 42d>≡ (68b)

```
#define PCRES 8
```

<struct COUNTER 42e>≡ (68b)

```
struct COUNTER
{
    char *name; /* function name */
    long time; /* ticks spent there */
};
```

<main() (kprof.c) display profile data 42f>≡ (41)

```
delta = data[0]-data[1];
print("total: %ld in kernel text: %ld outside kernel text: %ld\n",
    data[0], delta, data[1]);
if(data[0] == 0)
    exits(0);
if (!textsym(&s, 0))
    error(0, "no text symbols");

tbase = mach->kbase;
```

```

if(tbase != s.value & ~0xFFF)
    print("warning: kbase %.8llx != tbase %.8llx\n",
          tbase, s.value&~0xFFF);
print("KTZERO %.8llx PGSIZE %dKb\n", tbase, mach->pgsize/1024);
/*
 * Accumulate counts for each function
 */
cp = 0;
k = 0;
for (i = 0, j = 2; j < n; i++) {
    name = s.name; /* save name */
    if (!textsym(&s, i)) /* get next symbol */
        break;
    s.value -= tbase;
    s.value /= PCRES;
    sum = 0;
    while (j < n && j < s.value)
        sum += data[j++];
    if (sum) {
        cp = realloc(cp, (k+1)*sizeof(struct COUNTER));
        if (cp == 0)
            error(1, "realloc");
        cp[k].name = name;
        cp[k].time = sum;
        k++;
    }
}
if (!k)
    error(0, "no counts");
cp[k].time = 0; /* "etext" can take no time */
/*
 * Sort by time and print
 */
qsort(cp, k, sizeof(struct COUNTER), compar);
Binit(&outbuf, 1, OWRITE);
Bprint(&outbuf, "ms  %% sym\n");
while(--k>=0)
    Bprint(&outbuf, "%ld\t%3lld.%lld\t%s\n",
           cp[k].time,
           100LL*cp[k].time/delta,
           (1000LL*cp[k].time/delta)%10,
           cp[k].name);

```

<function error (misc/kprof.c) 43a> ≡ (68b)

```

static void
error(int perr, char *s)
{
    fprintf(2, "kprof: %s", s);
    if(perr){
        fprintf(2, ": ");
        perror(0);
    }else
        fprintf(2, "\n");
    exits(s);
}

```

<function compar 43b> ≡ (68b)

```

static int
compar(void *va, void *vb)
{

```

```
struct COUNTER *a, *b;

a = va;
b = vb;
if(a->time < b->time)
    return -1;
if(a->time == b->time)
    return 0;
return 1;
}
```

Chapter 9

Show real-time Behavior: /bin/trace

9.1 Data structures

```
<struct TEvent 45a>≡ (71b)
struct TEvent {
    Traceevent;
    vlong etime; /* length of block to draw */
};
```

```
<struct Task 45b>≡ (71b)
struct Task {
    int pid;
    char *name;
    int nevents;
    TEvent *events;
    vlong tstart;
    vlong total;
    vlong runtime;
    vlong runmax;
    vlong runthis;
    long runs;
    ulong tevents[Nevent];
};
```

```
<enum _anon_ (misc/trace.c) 45c>≡ (71b)
enum {
    Nevents = 1024,
    Ncolor = 6,
    K = 1024,
};
```

```
<struct scale 45d>≡ (71b)
struct scale {
    vlong scale;
    vlong bigtics;
    vlong littletics;
    int sleep;
};
```

```
<global scales 45e>≡ (71b)
struct scale scales[] = {
    { US(500), US(100), US(50), 0},
    { US(1000), US(500), US(100), 0},
    { US(2000), US(1000), US(200), 0},
    { US(5000), US(1000), US(500), 0},
};
```

```

{ MS(10), MS(5), MS(1), 20},
{ MS(20), MS(10), MS(2), 20},
{ MS(50), MS(10), MS(5), 20},
{ MS(100), MS(50), MS(10), 20}, /* starting scaleno */
{ MS(200), MS(100), MS(20), 20},
{ MS(500), MS(100), MS(50), 50},
{ MS(1000), MS(500), MS(100), 100},
{ MS(2000), MS(1000), MS(200), 100},
{ MS(5000), MS(1000), MS(500), 100},
{ S(10), S(50), S(1), 100},
{ S(20), S(10), S(2), 100},
{ S(50), S(10), S(5), 100},
{ S(100), S(50), S(10), 100},
{ S(200), S(100), S(20), 100},
{ S(500), S(100), S(50), 100},
{ S(1000), S(500), S(100), 100},
};

```

9.2 main()

<i><global verbose (misc/trace.c)</i> 46a	≡	(71b)
static int verbose;		
<i><global tasks</i> 46b	≡	(71b)
Task *tasks;		
<i><global cols</i> 46c	≡	(71b)
static Image *cols[Ncolor][4];		
<i><global profdev</i> 46d	≡	(71b)
char*profdev = "/proc/trace";		
<i><function usage</i> 46e	≡	(71b)
static void		
usage(void)		
{		
fprint(2, "Usage: %s [-d profdev] [-w] [-v] [-t triggerproc] [processes]\n", argv0);		
exits(nil);		
}		
<i><function threadmain</i> 46f	≡	(71b)
void		
threadmain(int argc, char **argv)		
{		
int fd, i;		
char fname[80];		
fmtinstall('t', timeconv);		
ARGBEGIN {		
case 'd':		
profdev = EARGF(usage());		
break;		
case 'v':		
verbose = 1;		
break;		
case 'w':		
newwin++;		
break;		

```

    case 't':
        triggerproc = (int)strtol(EARGF(usage()), nil, 0);
        break;
    default:
        usage();
}
ARGEND;

fname[sizeof fname - 1] = 0;
for(i = 0; i < argc; i++){
    snprintf(fname, sizeof fname - 2, "/proc/%s/ctl",
             argv[i]);
    if((fd = open(fname, OWRITE)) < 0){
        fprintf(2, "%s: cannot open %s: %r\n",
             argv[0], fname);
        continue;
    }

    if(fprint(fd, "trace 1") < 0)
        fprintf(2, "%s: cannot enable tracing on %s: %r\n",
             argv[0], fname);
    close(fd);
}

drawtrace();
}

⟨function mkcol 47a⟩≡ (71b)
static void
mkcol(int i, int c0, int c1, int c2)
{
    cols[i][0] = allocimagemix(display, c0, DWhite);
    cols[i][1] = allocimage(display, Rect(0,0,1,1), view->chan, 1, c1);
    cols[i][2] = allocimage(display, Rect(0,0,1,1), view->chan, 1, c2);
    cols[i][3] = allocimage(display, Rect(0,0,1,1), view->chan, 1, c0);
}

⟨function colinit 47b⟩≡ (71b)
static void
colinit(void)
{
    mediumfont = openfont(display, "/lib/font/bit/lucidasans/unicode.10.font");
    if(mediumfont == nil)
        mediumfont = font;
    tinyfont = openfont(display, "/lib/font/bit/lucidasans/unicode.7.font");
    if(tinyfont == nil)
        tinyfont = font;
    topmargin = mediumfont->height+2;
    bottommargin = tinyfont->height+2;

    /* Peach */
    mkcol(0, 0xFFAAAAFF, 0xFFAAAAFF, 0xBB5D5DFF);
    /* Aqua */
    mkcol(1, DPalebluegreen, DPalegreygreen, DPurpleblue);
    /* Yellow */
    mkcol(2, DPaleyellow, DDarkyellow, DYellowgreen);
    /* Green */
    mkcol(3, DPalegreen, DMedgreen, DDarkgreen);
    /* Blue */
    mkcol(4, 0x00AAFFFF, 0x00AAFFFF, 0x0088CCFF);
}

```



```

/* Grey */
cols[5][0] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xEEEEEEFF);
cols[5][1] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xCCCCCFF);
cols[5][2] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0x888888FF);
cols[5][3] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xAAAAAAFF);
grey = cols[5][2];
red = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xFF0000FF);
green = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0x00FF00FF);
blue = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0x0000FFFF);
bg = display->white;
fg = display->black;
}

```

```

⟨function time2x 48a⟩≡ (71b)
#define time2x(t) ((int)(((t) - oldestts) / ppp))

```

```

⟨function redraw 48b⟩≡ (71b)
static void
redraw(int scaleno)
{
    int n, i, j, x;
    char buf[256];
    Point p, q;
    Rectangle r, rtime;
    Task *t;
    vlong ts, oldestts, newestts, period, ppp, scale, s, ss;

    scale = scales[scaleno].scale;
    period = scale + scales[scaleno].littletics;
    ppp = period / Width; // period per pixel.

    /* Round 'now' to a nice number */
    newestts = now - (now % scales[scaleno].bigtics) +
        (scales[scaleno].littletics>>1);

    oldestts = newestts - period;

    //print("newestts %t, period %t, %d-%d\n", newestts, period, time2x(oldestts), time2x(newestts));
    if (prevts < oldestts){
        oldestts = newestts - period;

        prevts = oldestts;
        draw(view, view->r, bg, nil, ZP);
    }else{
        /* just white out time */
        rtime = view->r;
        rtime.min.x = rtime.max.x - stringwidth(mediumfont, "0000000000.000s");
        rtime.max.y = rtime.min.y + mediumfont->height;
        draw(view, rtime, bg, nil, ZP);
    }
    p = view->r.min;
    for (n = 0; n != ntasks; n++) {
        t = &tasks[n];
        /* p is upper left corner for this task */
        rtime = Rpt(p, addpt(p, Pt(500, mediumfont->height)));
        draw(view, rtime, bg, nil, ZP);
        snprintf(buf, sizeof(buf), "%d %s", t->pid, t->name);
        q = string(view, p, fg, ZP, mediumfont, buf);
        s = now - t->tstart;
        if(t->tevents[SRelease])

```

```

        snprintf(buf, sizeof(buf), " per %t | avg: %t max: %t",
            (vlong)(s/t->tevents[SRelease]),
            (vlong)(t->runtime/t->tevents[SRelease]),
            t->runmax);
    else if((s /=1000000000LL) != 0)
        snprintf(buf, sizeof(buf), " per 1s | avg: %t total: %t",
            t->total/s,
            t->total);
    else
        snprintf(buf, sizeof(buf), " total: %t", t->total);
    string(view, q, fg, ZP, tinyfont, buf);
    p.y += Height;
}
x = time2x(prevts);

p = view->r.min;
for (n = 0; n != ntasks; n++) {
    t = &tasks[n];

    /* p is upper left corner for this task */

    /* Move part already drawn */
    r = Rect(p.x, p.y + topmargin, p.x + x, p.y+Height);
    draw(view, r, view, nil, Pt(p.x + Width - x, p.y + topmargin));

    r.max.x = view->r.max.x;
    r.min.x += x;
    draw(view, r, bg, nil, ZP);

    line(view, addpt(p, Pt(x, Height - lineht)), Pt(view->r.max.x, p.y + Height - lineht),
        Endsquare, Endsquare, 0, cols[n % Ncolor][1], ZP);

    for (i = 0; i < t->nevents-1; i++)
        if (prevts < t->events[i + 1].time)
            break;

    if (i > 0) {
        memmove(t->events, t->events + i, (t->nevents - i) * sizeof(TEvent));
        t->nevents -= i;
    }

    for (i = 0; i != t->nevents; i++) {
        TEvent *e = &t->events[i], *_e;
        int sx, ex;

        switch (e->etype & 0xffff) {
        case SAdmit:
            if (e->time > prevts && e->time <= newestts) {
                sx = time2x(e->time);
                line(view, addpt(p, Pt(sx, topmargin)),
                    addpt(p, Pt(sx, Height - bottommargin)),
                    Endarrow, Endsquare, 1, green, ZP);
            }
            break;
        case SExpel:
            if (e->time > prevts && e->time <= newestts) {
                sx = time2x(e->time);
                line(view, addpt(p, Pt(sx, topmargin)),
                    addpt(p, Pt(sx, Height - bottommargin)),
                    Endsquare, Endarrow, 1, red, ZP);
            }
        }
    }
}

```

```

    }
    break;
case SRelease:
    if (e->time > prevts && e->time <= newestts) {
        sx = time2x(e->time);
        line(view, addpt(p, Pt(sx, topmargin)),
            addpt(p, Pt(sx, Height - bottommargin)),
            Endarrow, Endsquare, 1, fg, ZP);
    }
    break;
case SDeadline:
    if (e->time > prevts && e->time <= newestts) {
        sx = time2x(e->time);
        line(view, addpt(p, Pt(sx, topmargin)),
            addpt(p, Pt(sx, Height - bottommargin)),
            Endsquare, Endarrow, 1, fg, ZP);
    }
    break;

case SYield:
case SUser:
    if (e->time > prevts && e->time <= newestts) {
        sx = time2x(e->time);
        line(view, addpt(p, Pt(sx, topmargin)),
            addpt(p, Pt(sx, Height - bottommargin)),
            Endsquare, Endarrow, 0,
            (e->etype == SYield)? green: blue, ZP);
    }
    break;
case SSlice:
    if (e->time > prevts && e->time <= newestts) {
        sx = time2x(e->time);
        line(view, addpt(p, Pt(sx, topmargin)),
            addpt(p, Pt(sx, Height - bottommargin)),
            Endsquare, Endarrow, 0, red, ZP);
    }
    break;

case SRun:
    sx = time2x(e->time);
    ex = time2x(e->etime);
    if(ex == sx)
        ex++;

    r = Rect(sx, topmargin + 8, ex, Height - lineht);
    r = rectaddpt(r, p);

    draw(view, r, cols[n % Ncolor][e->etype==SRun?1:3], nil, ZP);

    if(t->pid == triggerproc && ex < Width)
        paused ^= 1;

    for(j = 0; j < t->nevents; j++){
        _e = &t->events[j];
        switch(_e->etype & 0xffff){
            case SInts:
                if (_e->time > prevts && _e->time <= newestts){
                    sx = time2x(_e->time);
                    line(view, addpt(p, Pt(sx, topmargin)),
                        addpt(p, Pt(sx, Height / 2 - bottommargin)),

```

```

                                Endsquare, Endsquare, 0,
                                green, ZP);
        }
        break;
    case SInte:
        if (_e->time > prevts && _e->time <= newestts) {
            sx = time2x(_e->time);
            line(view, addpt(p, Pt(sx, Height / 2 - bottommargin)),
                addpt(p, Pt(sx, Height - bottommargin)),
                Endsquare, Endsquare, 0,
                blue, ZP);
        }
        break;
    }
    break;
}
p.y += Height;
}

ts = prevts + scales[scaleno].littletics - (prevts % scales[scaleno].littletics);
x = time2x(ts);

while(x < Width){
    p = view->r.min;
    for(n = 0; n < ntasks; n++){
        int height, width;

        /* p is upper left corner for this task */
        if ((ts % scales[scaleno].scale) == 0){
            height = 10 * Height;
            width = 1;
        }else if ((ts % scales[scaleno].bigtics) == 0){
            height = 12 * Height;
            width = 0;
        }else{
            height = 13 * Height;
            width = 0;
        }
        height >>= 4;

        line(view, addpt(p, Pt(x, height)), addpt(p, Pt(x, Height - lineht)),
            Endsquare, Endsquare, width, cols[n % Ncolor][2], ZP);

        p.y += Height;
    }
    ts += scales[scaleno].littletics;
    x = time2x(ts);
}

rtime = view->r;
rtime.min.y = rtime.max.y - tinyfont->height + 2;
draw(view, rtime, bg, nil, ZP);
ts = oldestts + scales[scaleno].bigtics - (oldestts % scales[scaleno].bigtics);
x = time2x(ts);
ss = 0;
while(x < Width){
    snprintf(buf, sizeof(buf), "%t", ss);
    string(view, addpt(p, Pt(x - stringwidth(tinyfont, buf)/2, - tinyfont->height - 1)),

```

```

        fg, ZP, tinyfont, buf);
    ts += scales[scalesno].bigtics;
    ss += scales[scalesno].bigtics;
    x = time2x(ts);
}

snprintf(buf, sizeof(buf), "%t", now);
string(view, Pt(view->r.max.x - stringwidth(mediumfont, buf), view->r.min.y),
        fg, ZP, mediumfont, buf);

flushimage(display, 1);
prevts = newestts;
}

```

<function newtask 52a>≡ (71b)

```

Task*
newtask(ulong pid)
{
    Task *t;
    char buf[64], *p;
    int fd,n;

    tasks = realloc(tasks, (ntasks + 1) * sizeof(Task));
    assert(tasks);

    t = &tasks[ntasks++];
    memset(t, 0, sizeof(Task));
    t->events = nil;
    snprintf(buf, sizeof buf, "/proc/%ld/status", pid);
    t->name = nil;
    fd = open(buf, OREAD);
    if (fd >= 0){
        n = read(fd, buf, sizeof buf);
        if(n > 0){
            p = buf + sizeof buf - 1;
            *p = 0;
            p = strchr(buf, ' ');
            if (p) *p = 0;
            t->name = strdup(buf);
        }else
            print("%s: %r\n", buf);
        close(fd);
    }else
        print("%s: %r\n", buf);
    t->pid = pid;
    prevts = 0;
    if (newwin){
        fprintf(wctlfd, "resize -dx %d -dy %d\n",
            Width + 20, (ntasks * Height) + 5);
    }else
        Height = ntasks ? Dy(view->r)/ntasks : Dy(view->r);
    return t;
}

```

<function doevent 52b>≡ (71b)

```

void
doevent(Task *t, Traceevent *ep)
{
    int i, n;
    TEvent *event;

```

```

vlong runt;

t->tevents[ep->etype & 0xffff]++;
n = t->nevents++;
t->events = realloc(t->events, t->nevents*sizeof(TEvent));
assert(t->events);
event = &t->events[n];
memmove(event, ep, sizeof(Traceevent));
event->etime = 0;

switch(event->etype & 0xffff){
case SRelease:
    if (t->runthis > t->runmax)
        t->runmax = t->runthis;
    t->runthis = 0;
    break;

case SSleep:
case SYield:
case SReady:
case SSlice:
    for(i = n-1; i >= 0; i--){
        if (t->events[i].etype == SRun)
            break;
    }
    if(i < 0 || t->events[i].etime != 0)
        break;
    runt = event->time - t->events[i].time;
    if(runt > 0){
        t->events[i].etime = event->time;
        t->runtime += runt;
        t->total += runt;
        t->runthis += runt;
        t->runs++;
    }
    break;
case SDead:
    print("task died %ld %t %s\n", event->pid, event->time, schedstatename[event->etype & 0xffff]);
    free(t->events);
    free(t->name);
    ntasks--;
    memmove(t, t+1, sizeof(Task)*(ntasks-1));
    if (newwin)
        fprintf(wctlf, "resize -dx %d -dy %d\n",
            Width + 20, (ntasks * Height) + 5);
    else
        Height = ntasks ? Dy(view->r)/ntasks : Dy(view->r);
    prevts = 0;
}
}

```

<function drawtrace 53>≡

(71b)

```

void
drawtrace(void)
{
    char *wsys, line[256];
    int wfd, logfd;
    Mousectl *mousectl;
    Keyboardctl *keyboardctl;
    int scaleno;
    Rune r;

```

```

int i, n;
Task *t;
Traceevent *ep;

eventbuf = malloc(Nevents*sizeof(Traceevent));
assert(eventbuf);

if((logfd = open(profdev, OREAD)) < 0)
    sysfatal("%s: Cannot open %s: %r", argv0, profdev);

if(newwin){
    if((wsys = getenv("wsys")) == nil)
        sysfatal("%s: Cannot find windowing system: %r",
            argv0);

    if((wfd = open(wsys, ORDWR)) < 0)
        sysfatal("%s: Cannot open windowing system: %r",
            argv0);

    snprintf(line, sizeof(line), "new -pid %d -dx %d -dy %d",
        getpid(), Width + 20, Height + 5);
    line[sizeof(line) - 1] = '\0';
    rfork(RFNAMEG);

    if(mount(wfd, -1, "/mnt/wsys", MREPL, line) < 0)
        sysfatal("%s: Cannot mount %s under /mnt/wsys: %r",
            argv0, line);

    if(bind("/mnt/wsys", "/dev", MBEFORE) < 0)
        sysfatal("%s: Cannot bind /mnt/wsys in /dev: %r",
            argv0);

}
if((wctlfd = open("/dev/wctl", OWRITE)) < 0)
    sysfatal("%s: Cannot open /dev/wctl: %r", argv0);
if(initdraw(nil, nil, "trace") < 0)
    sysfatal("%s: initdraw failure: %r", argv0);

Width = Dx(view->r);
Height = Dy(view->r);

if((mousectl = initmouse(nil, view)) == nil)
    sysfatal("%s: cannot initialize mouse: %r", argv0);

if((keyboardctl = initkeyboard(nil)) == nil)
    sysfatal("%s: cannot initialize keyboard: %r", argv0);

colinit();

paused = 0;
scaleno = 7; /* 100 milliseconds */
now = nsec();
for(;;) {
    Alt a[] = {
        { mousectl->c, nil, CHANRCV },
        { mousectl->resizec, nil, CHANRCV },
        { keyboardctl->c, &r, CHANRCV },
        { nil, nil, CHANNOBLK },
    };
};

```

```

switch (alt(a)) {
case 0:
    continue;

case 1:
    if(getwindow(display, Refnone) < 0)
        sysfatal("drawrt: Cannot re-attach window");
    if(newwin){
        if(Dx(view->r) != Width ||
            Dy(view->r) != (ntasks * Height)){
            fprintf(2, "resize: x: have %d, need %d; y: have %d, need %d\n",
                Dx(view->r), Width + 8, Dy(view->r), (ntasks * Height) + 8);
            fprintf(wctlfd, "resize -dx %d -dy %d\n",
                Width + 8, (ntasks * Height) + 8);
        }
    }
    else{
        Width = Dx(view->r);
        Height = ntasks? Dy(view->r)/ntasks:
            Dy(view->r);
    }
    break;

case 2:

    switch(r){
    case 'r':
        for(i = 0; i < ntasks; i++){
            tasks[i].tstart = now;
            tasks[i].total = 0;
            tasks[i].runtime = 0;
            tasks[i].runmax = 0;
            tasks[i].runthis = 0;
            tasks[i].runs = 0;
            memset(tasks[i].tevents, 0, Nevent*sizeof(ulong));
        }
        break;

    case 'p':
        paused ^= 1;
        prevts = 0;
        break;

    case '-':
        if (scaleno < nelem(scales) - 1)
            scaleno++;
        prevts = 0;
        break;

    case '+':
        if (scaleno > 0)
            scaleno--;
        prevts = 0;
        break;

    case 'q':
        threadexitsall(nil);

    case 'v':

```



```

        verbose ^= 1;

    default:
        break;
    }
    break;

case 3:
    now = nsec();
    while((n = read(logfd, eventbuf, Nevents*sizeof(Traceevent))) > 0){
        assert((n % sizeof(Traceevent)) == 0);
        nevents = n / sizeof(Traceevent);
        for (ep = eventbuf; ep < eventbuf + nevents; ep++){
            if ((ep->etype & 0xffff) >= Nevent){
                print("%ld %t Illegal event %ld\n",
                    ep->pid, ep->time, ep->etype & 0xffff);
                continue;
            }
            if (verbose)
                print("%ld %t %s\n",
                    ep->pid, ep->time, schedstatename[ep->etype & 0xffff]);

            for(i = 0; i < ntasks; i++)
                if(tasks[i].pid == ep->pid)
                    break;

            if(i == ntasks){
                t = newtask(ep->pid);
                t->tstart = ep->time;
            }else
                t = &tasks[i];

            doevent(t, ep);
        }
    }
    if(!paused)
        redraw(scaleno);
}
sleep(scales[scaleno].sleep);
}
}

```

```

⟨function timeconv 56⟩≡ (71b)
int
timeconv(Fmt *f)
{
    char buf[128], *sign;
    vlong t;

    buf[0] = 0;
    switch(f->r) {
    case 'U':
        t = va_arg(f->args, vlong);
        break;
    case 't': // vlong in nanoseconds
        t = va_arg(f->args, vlong);
        break;
    default:
        return fmtstrcpy(f, "(timeconv)");
    }
}

```

```

if (t < 0) {
    sign = "-";
    t = -t;
}else
    sign = "";
if (t > S(1)){
    t += OneRound;
    sprintf(buf, "%s%d.%.3ds", sign, (int)(t / S(1)), (int)(t % S(1))/1000000);
}else if (t > MS(1)){
    t += MilliRound;
    sprintf(buf, "%s%d.%.3dms", sign, (int)(t / MS(1)), (int)(t % MS(1))/1000);
}else if (t > US(1))
    sprintf(buf, "%s%d.%.3dus", sign, (int)(t / US(1)), (int)(t % US(1)));
else
    sprintf(buf, "%s%dns", sign, (int)t);
return fmtstrcpy(f, buf);
}

```

Chapter 10

System Monitoring /bin/stats

10.1 Data structures

```
<constant MAXNUM 58a>≡ (92)
// a GUI system monitoring tool
```

```
#define MAXNUM 10 /* maximum number of numbers on data line */
```

```
<struct Graph 58b>≡ (92)
struct Graph
{
    int colindex;
    Rectangle r;
    uulong *data;
    int ndata;
    char *label;
    void (*newvalue)(Machine*, uulong*, uulong*, int);
    void (*update)(Graph*, uulong, uulong);
    Machine *mach;
    int overflow;
    Image *overtmp;
};
```

```
<enum _anon_ (misc/stats.c) 58c>≡ (92)
enum
{
    /* old /dev/swap */
    Mem = 0,
    Maxmem,
    Swap,
    Maxswap,

    /* /dev/sysstats */
    Procno = 0,
    Context,
    Interrupt,
    Syscall,
    Fault,
    TLBfault,
    TLBpurge,
    Load,
    Idle,
    InIntr,
    /* /net/ether0/stats */
    In = 0,
```

```

    Link,
    Out,
    Err0,
};

⟨struct Machine 59a⟩≡ (92)
struct Machine
{
    char *name;
    char *shortname;
    int remote;
    int statsfd;
    int swapfd;
    int etherfd;
    int ifstatsfd;
    int batteryfd;
    int bitsybatfd;
    int tempfd;
    int disable;

    uvlong devswap[4];
    uvlong devsysstat[10];
    uvlong prevsysstat[10];
    int nproc;
    int lgproc;
    uvlong netetherstats[8];
    uvlong prevetherstats[8];
    uvlong batterystats[2];
    uvlong netetherifstats[2];
    uvlong temp[10];

    /* big enough to hold /dev/sysstat even with many processors */
    char buf[8*1024];
    char *bufp;
    char *ebufp;
};

⟨enum _anon_ (misc/stats.c)2 59b⟩≡ (92)
enum
{
    Mainproc,
    Mouseproc,
    NPROC,
};

⟨enum _anon_ (misc/stats.c)3 59c⟩≡ (92)
enum
{
    Ncolor = 6,
    Ysqueeze = 2, /* vertical squeezing of label text */
    Labspace = 2, /* room around label */
    Dot = 2, /* height of dot */
    Opwid = 5, /* strlen("add ") or strlen("drop ") */
    Nlab = 3, /* max number of labels on y axis */
    Lablen = 16, /* max length of label */
    Lx = 4, /* label tick length */
};

```

```

⟨enum Menu2 60a⟩≡ (92)
enum Menu2
{
    Mbattery,
    Mcontext,
    Mether,
    Methererr,
    Metherin,
    Metherout,
    Mfault,
    Midle,
    Minintr,
    Mintr,
    Mload,
    Mmem,
    Mswap,
    Msyscall,
    Mtlbmiss,
    Mtlbpurge,
    Msignal,
    Mtemp,
    Nmenu2,
};

```

```

⟨global menu2str 60b⟩≡ (92)
char *menu2str[Nmenu2+1] = {
    "add battery ",
    "add context ",
    "add ether  ",
    "add ethererr",
    "add etherin ",
    "add etherout",
    "add fault  ",
    "add idle   ",
    "add inintr ",
    "add intr   ",
    "add load   ",
    "add mem    ",
    "add swap   ",
    "add syscall ",
    "add tlbmiss ",
    "add tlbpurge",
    "add 802.11b ",
    "add temp    ",
    nil,
};

```

```

⟨global menu2 60c⟩≡ (92)
Menu menu2 = {menu2str, nil};

```

```

⟨global present 60d⟩≡ (92)
int present[Nmenu2];

```

```

⟨global newvaluefn 60e⟩≡ (92)
void (*newvaluefn[Nmenu2])(Machine*, uvlong*, uvlong*, int init) = {
    batteryval,
    contextval,
    etherval,
    ethererrval,
    etherinval,
};

```

```

    etheroutval,
    faultval,
    idleval,
    inintrval,
    intrval,
    loadval,
    memval,
    swapval,
    syscallval,
    tlbmissval,
    tlbpurgeval,
    signalval,
    tempval,
};

⟨global cols (misc/stats.c) 61a⟩≡ (92)
    Image *cols[Ncolor][3];

⟨global graph 61b⟩≡ (92)
    Graph *graph;

⟨global mach 61c⟩≡ (92)
    Machine *mach;

⟨global mediumfont 61d⟩≡ (92)
    Font *mediumfont;

⟨global mysysname 61e⟩≡ (92)
    char *mysysname;

⟨global argchars 61f⟩≡ (92)
    char argchars[] = "8bceEfiImlnpstwz";

⟨global pids 61g⟩≡ (92)
    int pids[NPROC];

⟨global parity 61h⟩≡ (92)
    int parity; /* toggled to avoid patterns in textured background */

⟨global nmach 61i⟩≡ (92)
    int nmach;

⟨global ngraph 61j⟩≡ (92)
    int ngraph; /* totaly number is ngraph*nmach */

⟨global scale 61k⟩≡ (92)
    double scale = 1.0;

⟨global logscale 61l⟩≡ (92)
    int logscale = 0;

⟨global ylabels 61m⟩≡ (92)
    int ylabels = 0;

⟨global oldsystem 61n⟩≡ (92)
    int oldsystem = 0;

⟨global sleeptime 61o⟩≡ (92)
    int sleeptime = 1000;

```

```

⟨global procnames 62a⟩≡ (92)
char *procnames[NPROC] = {"main", "mouse"};

```

10.2 main()

```

⟨function main (misc/stats.c) 62b⟩≡ (92)
void
main(int argc, char *argv[])
{
    int i, j;
    double secs;
    uulong v, vmax, nargs;
    char args[100];

    nmach = 1;
    mysysname = getenv("sysname");
    if(mysysname == nil){
        fprintf(2, "stats: can't find $sysname: %r\n");
        exits("sysname");
    }
    mysysname = estrdup(mysysname);

    nargs = 0;
    ARGBEGIN{
    case 'T':
        secs = atof(EARGF(usage()));
        if(secs > 0)
            sleeptime = 1000*secs;
        break;
    case 'S':
        scale = atof(EARGF(usage()));
        if(scale <= 0)
            usage();
        break;
    case 'L':
        logscale++;
        break;
    case 'Y':
        ylabels++;
        break;
    case 'O':
        oldsystem = 1;
        break;
    default:
        if(nargs>=sizeof args || strchr(argchars, ARGC())==nil)
            usage();
        args[nargs++] = ARGC();
    }ARGEND

    if(argc == 0){
        mach = emalloc(nmach*sizeof(Machine));
        initmach(&mach[0], mysysname);
        readmach(&mach[0], 1);
    }else{
        for(i=j=0; i<argc; i++){
            if (addmachine(argv[i]))
                readmach(&mach[j++], 1);
        }
    }
}

```

```

    if (j == 0)
        exits("connect");
}

for(i=0; i<nargs; i++)
switch(args[i]){
default:
    fprintf(2, "stats: internal error: unknown arg %c\n", args[i]);
    usage();
case 'b':
    addgraph(Mbattery);
    break;
case 'c':
    addgraph(Mcontext);
    break;
case 'e':
    addgraph(Mether);
    break;
case 'E':
    addgraph(Metherin);
    addgraph(Metherout);
    break;
case 'f':
    addgraph(Mfault);
    break;
case 'i':
    addgraph(Mintr);
    break;
case 'I':
    addgraph(Mload);
    addgraph(Midle);
    addgraph(Minintr);
    break;
case 'l':
    addgraph(Mload);
    break;
case 'm':
    addgraph(Mmem);
    break;
case 'n':
    addgraph(Metherin);
    addgraph(Metherout);
    addgraph(Methererr);
    break;
case 'p':
    addgraph(Mtlbpurge);
    break;
case 's':
    addgraph(Msyscall);
    break;
case 't':
    addgraph(Mtlbmiss);
    addgraph(Mtlbpurge);
    break;
case '8':
    addgraph(Msignal);
    break;
case 'w':
    addgraph(Mswap);
    break;

```



```

case 'z':
    addgraph(Mtemp);
    break;
}

if(ngraph == 0)
    addgraph(Mload);

for(i=0; i<nmach; i++)
    for(j=0; j<ngraph; j++)
        graph[i*ngraph+j].mach = &mach[i];

if(initdraw(nil, nil, "stats") < 0){
    fprintf(2, "stats: initdraw failed: %r\n");
    exits("initdraw");
}
colinit();
einit(Emouse);
notify(nil);
startproc(mouseproc, Mouseproc);
pids[Mainproc] = getpid();
display->locking = 1; /* tell library we're using the display lock */

resize();

unlockdisplay(display); /* display is still locked from initdraw() */
for(;;){
    for(i=0; i<nmach; i++)
        readmach(&mach[i], 0);
    lockdisplay(display);
    parity = 1-parity;
    for(i=0; i<nmach*ngraph; i++){
        graph[i].newvalue(graph[i].mach, &v, &vmax, 0);
        graph[i].update(&graph[i], v, vmax);
    }
    flushimage(display, 1);
    unlockdisplay(display);
    sleep(sleeptime);
}
}

```

Chapter 11

IO Monitoring: `/bin/iostats`

Chapter 12

Conclusion

You now know how the Plan 9 profiling tools work, to the smallest details, and more generally how many profilers work.

Despite totaling only about 4900 lines of C, these tools cover the three main approaches to profiling: instrumentation (**prof**, via the linker's `_profin/_profout` hooks), sampling (**tprof** and **kprof**, via the kernel's `profclock()` timer), and tracing/monitoring (**trace** and **stats**, via `/proc/trace` and kernel pseudo-files). The simplest tool, **time**, is barely 60 lines. The most complex, **stats**, reaches about 1600 lines, mostly because of its graphical display code. Along the way, you have seen how profiling depends on the cooperation of three layers: the kernel (time accounting, PC sampling, scheduler events), the linker (function call instrumentation), and user-space tools (reading and presenting the data). You have also seen the **Tos** structure, an elegant mechanism that lets user code read precise timing data without the overhead of a system call.

12.1 Connections to other books

- **KERNEL** book [Pad14] describes the kernel-side infrastructure that the profiling tools rely on: `accounttime()` for per-process time tracking, `profclock()` and the `profile` array for PC sampling, `/proc/trace` for scheduler events, and the **Tos** structure for cycle counting. The kernel's `kprof(3)` device provides kernel self-profiling.
- **LINKER** book [Pad15] explains how `5l -p` instruments programs by inserting `BL _profin/BL _profout` around every function, and how `5l -p -1` implements the simpler `__mcount` call-counting strategy.
- **LIBCORE** book [Pad16a] covers `libmach`, the library that **prof**, **tprof**, and **kprof** use to read symbol tables from executables and map program counter values to function names.
- **GRAPHICS** book [Pad16b] covers the draw library used by **trace** and **stats** for their graphical displays.
- **SHELL** book [Pad18] explains the `rc` shell, including the `$apid` variable used in the **tprof** workflow and the `rfork/exec/wait` model that **time** relies on.

12.2 Beyond the Plan 9 profilers

The Plan 9 profilers are intentionally simple. Here are some features found in more advanced profiling tools that are not present in the Plan 9 suite:

- *Hardware performance counters*: modern CPUs provide counters for cache misses, branch mispredictions, TLB misses, etc. Tools like Linux's **perf** can read these to give a much richer picture than time alone.

- *Call graph from sampling*: some profilers (e.g., **perf**) walk the stack at each sample to reconstruct call graphs from sampling data, combining the low overhead of sampling with the structural information of instrumentation.
- *Flame graphs*: a visualization technique (invented by Brendan Gregg) that displays sampled call stacks as nested rectangles, making it easy to spot which call paths consume the most time.
- *Dynamic instrumentation*: tools like DTrace and SystemTap can instrument arbitrary points in a running program or kernel without recompilation, by patching instructions at runtime.

Despite these limitations, the Plan 9 profilers demonstrate that effective profiling does not require enormous complexity. A few hundred lines of C, combined with simple kernel support, are enough to answer the most common performance questions: where is my program spending its time, and how many times is each function called?

Appendix A

Extra Code

A.1 profilers/

A.1.1 misc/time.c

```
<misc/time.c 68a>≡  
#include <u.h>  
#include <libc.h>  
  
<global output(time.c) 23a>  
  
//forward decl  
void add(char*, ...);  
void notifyf(void*, char*);  
  
<function error(time.c) 23e>  
<function main(time.c) 22>  
<function add(time.c) 23b>  
<function notifyf(time.c) 24b>
```

A.1.2 misc/kprof.c

```
<misc/kprof.c 68b>≡  
#include <u.h>  
#include <libc.h>  
#include <bio.h>  
#include <mach.h>  
  
<constant PCRES 42d>  
  
<struct COUNTER 42e>  
  
<function error (misc/kprof.c) 43a>  
  
<function compar 43b>  
<function main (misc/kprof.c) 41>
```

A.1.3 misc/prof.c

```
<misc/prof.c 68c>≡  
#include <u.h>  
#include <libc.h>  
#include <bio.h>
```

```

#include <mach.h>

typedef struct Data Data;
typedef struct Pc Pc;
typedef struct Acc Acc;

⟨struct Data 25a⟩

⟨struct Pc 26c⟩

⟨struct Acc 26a⟩

⟨global data 25b⟩
⟨global acc 26b⟩
⟨global ms 30a⟩
⟨global nsym 28a⟩
⟨global ndata 25c⟩
⟨global dflag 32b⟩
⟨global rflag 34d⟩
⟨global bout 27a⟩
⟨global tabstop 34a⟩
⟨global verbose 27b⟩

void syms(char*);
void datas(char*);
void graph(int, ulong, Pc*);
void plot(void);
char* name(ulong);
void indent(int);
char* defaout(void);

⟨function main (misc/prof.c) 26d⟩

⟨function swapdata 29a⟩

⟨function acmp 32a⟩

⟨function syms 28b⟩

⟨function datas 28e⟩

⟨function name 33c⟩

⟨function graph 33a⟩
⟨function symind 31c⟩

⟨function sum 30c⟩

⟨function plot 29e⟩

⟨function indent 34c⟩

⟨global trans 27e⟩

⟨function defaout 27d⟩

```

A.1.4 misc/trace.c

⟨function NS 69⟩≡

(71b)

```

#define NS(x) ((vlong)x)

⟨function US 70a⟩≡ (71b)
#define US(x) (NS(x) * 1000ULL)

⟨function MS 70b⟩≡ (71b)
#define MS(x) (US(x) * 1000ULL)

⟨function S 70c⟩≡ (71b)
#define S(x) (MS(x) * 1000ULL)

⟨function numblocks 70d⟩≡ (71b)
#define numblocks(a, b) (((a) + (b) - 1) / (b))

⟨function roundup 70e⟩≡ (71b)
#define roundup(a, b) (numblocks((a), (b)) * (b))

⟨enum _anon_ 70f⟩≡ (71b)
enum {
    OneRound = MS(1)/2LL,
    MilliRound = US(1)/2LL,
};

⟨global newwin 70g⟩≡ (71b)
int newwin;

⟨global Width 70h⟩≡ (71b)
int Width = 1000;

⟨global Height 70i⟩≡ (71b)
int Height = 100; // Per task

⟨global topmargin 70j⟩≡ (71b)
int topmargin = 8;

⟨global bottommargin 70k⟩≡ (71b)
int bottommargin = 4;

⟨global lineht 70l⟩≡ (71b)
int lineht = 12;

⟨global wctlfd 70m⟩≡ (71b)
int wctlfd;

⟨global nevents 70n⟩≡ (71b)
int nevents;

⟨global eventbuf 70o⟩≡ (71b)
Traceevent *eventbuf;

⟨global event 70p⟩≡ (71b)
TEvent *event;

```

<global schedstatename 71a>≡ (71b)

```
char *schedstatename[] = {
    [SReady] = "Ready",
    [SRun] = "Run",
    [SDead] = "Dead",
    [SSleep] = "Sleep",
    [SUser] = "User",

    [SAdmit] = "Admit",
    [SRelease] = "Release",
    [SYield] = "Yield",
    [SSlice] = "Slice",
    [SDeadline] = "Deadline",
    [SExpel] = "Expel",
    [SInts] = "Ints",
    [SInte] = "Inte",
};
```

<misc/trace.c 71b>≡

```
#include <u.h>
#include <tos.h>
#include <libc.h>
#include <thread.h>
#include <ip.h>
#include <bio.h>

#include <draw.h>
#include <window.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>

#include <trace.h>
```

```
// a GUI tracer for kernel scheduler events
```

```
#pragma varargck type "t" vlong
#pragma varargck type "U" uulong
```

```
<function NS 69>
<function US 70a>
<function MS 70b>
<function S 70c>
```

```
<function numblocks 70d>
<function roundup 70e>
```

```
<enum _anon_ 70f>
```

```
typedef struct TEvent TEvent;
typedef struct Task Task;
<struct TEvent 45a>
```

```
<struct Task 45b>
```

```
<enum _anon_ (misc/trace.c) 45c>
```

```
vlong now, prevts;
```

```
<global newwin 70g>
```



```

<global Width 70h>
<global Height 70i>
<global topmargin 70j>
<global bottommargin 70k>
<global lineht 70l>
<global wctlfd 70m>
<global nevents 70n>
<global eventbuf 70o>
<global event 70p>

void drawtrace(void);
int schedparse(char*, char*, char*);
int timeconv(Fmt*);

<global schedstatename 71a>

<struct scale 45d>

<global scales 45e>

int ntasks, triggerproc, paused;
<global verbose (misc/trace.c) 46a>
<global tasks 46b>
<global cols 46c>
static Font *mediumfont, *tinyfont;
Image *grey, *red, *green, *blue, *bg, *fg;
<global profdev 46d>

<function usage 46e>

<function threadmain 46f>

<function mkcol 47a>

<function colinit 47b>

<function time2x 48a>

<function redraw 48b>

<function newtask 52a>

<function doevent 52b>

<function drawtrace 53>

<function timeconv 56>

```

A.1.5 misc/tprof.c

```

<misc/tprof.c 72>≡
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <mach.h>

<constant PCRES (misc/tprof.c) 38b>

<struct COUNTER (misc/tprof.c) 38c>

```

<function error (misc/tprof.c) 39b>

<function compar (misc/tprof.c) 39a>

<function main (misc/tprof.c) 36>

A.1.6 misc/stats.c

<function killall 73a>≡ (92)

```
void
killall(char *s)
{
    int i, pid;

    pid = getpid();
    for(i=0; i<NPROC; i++)
        if(pids[i] && pids[i]!=pid)
            postnote(PNPROC, pids[i], "kill");
    exits(s);
}
```

<function emalloc 73b>≡ (92)

```
void*
emalloc(ulong sz)
{
    void *v;
    v = malloc(sz);
    if(v == nil) {
        fprintf(2, "stats: out of memory allocating %ld: %r\n", sz);
        killall("mem");
    }
    memset(v, 0, sz);
    return v;
}
```

<function erealloc 73c>≡ (92)

```
void*
erealloc(void *v, ulong sz)
{
    v = realloc(v, sz);
    if(v == nil) {
        fprintf(2, "stats: out of memory reallocating %ld: %r\n", sz);
        killall("mem");
    }
    return v;
}
```

<function estrdup 73d>≡ (92)

```
char*
estrdup(char *s)
{
    char *t;
    if((t = strdup(s)) == nil) {
        fprintf(2, "stats: out of memory in strdup(%.10s): %r\n", s);
        killall("mem");
    }
    return t;
}
```

<function mkcol (misc/stats.c) 74a>≡ (92)

```
void
mkcol(int i, int c0, int c1, int c2)
{
    cols[i][0] = allocimagemix(display, c0, DWhite);
    cols[i][1] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, c1);
    cols[i][2] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, c2);
}
```

<function colinit (misc/stats.c) 74b>≡ (92)

```
void
colinit(void)
{
    mediumfont = openfont(display, "/lib/font/bit/pelm/latin1.8.font");
    if(mediumfont == nil)
        mediumfont = font;

    /* Peach */
    mkcol(0, 0xFFAAAAFF, 0xFFAAAAFF, 0xBB5D5DFF);
    /* Aqua */
    mkcol(1, DPalebluegreen, DPalegreygreen, DPurpleblue);
    /* Yellow */
    mkcol(2, DPaleyellow, DDarkyellow, DYellowgreen);
    /* Green */
    mkcol(3, DPalegreen, DMedgreen, DDarkgreen);
    /* Blue */
    mkcol(4, 0x00AAFFFF, 0x00AAFFFF, 0x0088CCFF);
    /* Grey */
    cols[5][0] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, 0xEEEEEEFF);
    cols[5][1] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, 0xCCCCCFF);
    cols[5][2] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, 0x888888FF);
}
```

<function loadbuf 74c>≡ (92)

```
int
loadbuf(Machine *m, int *fd)
{
    int n;

    if(*fd < 0)
        return 0;
    seek(*fd, 0, 0);
    n = read(*fd, m->buf, sizeof m->buf-1);
    if(n <= 0){
        close(*fd);
        *fd = -1;
        return 0;
    }
    m->bufp = m->buf;
    m->ebufp = m->buf+n;
    m->buf[n] = 0;
    return 1;
}
```

<function label 74d>≡ (92)

```
void
label(Point p, int dy, char *text)
{
    char *s;
```

```

Rune r[2];
int w, maxw, maxy;

p.x += Labspace;
maxy = p.y+dy;
maxw = 0;
r[1] = '\0';
for(s=text; *s; ){
    if(p.y+mediumfont->height-Ysqueeze > maxy)
        break;
    w = chartorune(r, s);
    s += w;
    w = runestringwidth(mediumfont, r);
    if(w > maxw)
        maxw = w;
    runestring(view, p, display->black, ZP, mediumfont, r);
    p.y += mediumfont->height-Ysqueeze;
}
}

⟨function paritypt 75a⟩≡ (92)
Point
paritypt(int x)
{
    return Pt(x+parity, 0);
}

⟨function datapoint 75b⟩≡ (92)
Point
datapoint(Graph *g, int x, uvlong v, uvlong vmax)
{
    Point p;
    double y;

    p.x = x;
    y = ((double)v)/(vmax*scale);
    if(logscale){
        /*
         * Arrange scale to cover a factor of 1000.
         * vmax corresponds to the 100 mark.
         * 10*vmax is the top of the scale.
         */
        if(y <= 0.)
            y = 0;
        else{
            y = log10(y);
            /* 1 now corresponds to the top; -2 to the bottom; rescale */
            y = (y+2.)/3.;
        }
    }
    if(y < 0x7fffffff){ /* avoid floating overflow */
        p.y = g->r.max.y - Dy(g->r)*y - Dot;
        if(p.y < g->r.min.y)
            p.y = g->r.min.y;
        if(p.y > g->r.max.y-Dot)
            p.y = g->r.max.y-Dot;
    }else
        p.y = g->r.max.y-Dot;
    return p;
}

```

<function drawdatum 76a>≡ (92)

```
void
drawdatum(Graph *g, int x, uvlong prev, uvlong v, uvlong vmax)
{
    int c;
    Point p, q;

    c = g->colindex;
    p = datapoint(g, x, v, vmax);
    q = datapoint(g, x, prev, vmax);
    if(p.y < q.y){
        draw(view, Rect(p.x, g->r.min.y, p.x+1, p.y), cols[c][0], nil, paritypt(p.x));
        draw(view, Rect(p.x, p.y, p.x+1, q.y+Dot), cols[c][2], nil, ZP);
        draw(view, Rect(p.x, q.y+Dot, p.x+1, g->r.max.y), cols[c][1], nil, ZP);
    }else{
        draw(view, Rect(p.x, g->r.min.y, p.x+1, q.y), cols[c][0], nil, paritypt(p.x));
        draw(view, Rect(p.x, q.y, p.x+1, p.y+Dot), cols[c][2], nil, ZP);
        draw(view, Rect(p.x, p.y+Dot, p.x+1, g->r.max.y), cols[c][1], nil, ZP);
    }
}
```

<function redraw (misc/stats.c) 76b>≡ (92)

```
void
redraw(Graph *g, uvlong vmax)
{
    int i, c;

    c = g->colindex;
    draw(view, g->r, cols[c][0], nil, paritypt(g->r.min.x));
    for(i=1; i<Dx(g->r); i++){
        drawdatum(g, g->r.max.x-i, g->data[i-1], g->data[i], vmax);
        drawdatum(g, g->r.min.x, g->data[i], g->data[i], vmax);
        g->overflow = 0;
    }
}
```

<function update1 76c>≡ (92)

```
void
update1(Graph *g, uvlong v, uvlong vmax)
{
    char buf[48];
    int overflow;

    if(g->overflow && g->overtmp!=nil)
        draw(view, g->overtmp->r, g->overtmp, nil, g->overtmp->r.min);
    draw(view, g->r, view, nil, Pt(g->r.min.x+1, g->r.min.y));
    drawdatum(g, g->r.max.x-1, g->data[0], v, vmax);
    memmove(g->data+1, g->data, (g->ndata-1)*sizeof(g->data[0]));
    g->data[0] = v;
    g->overflow = 0;
    if(logscale)
        overflow = (v>10*vmax*scale);
    else
        overflow = (v>vmax*scale);
    if(overflow && g->overtmp!=nil){
        g->overflow = 1;
        draw(g->overtmp, g->overtmp->r, view, nil, g->overtmp->r.min);
        sprintf(buf, "%lld", v);
        string(view, g->overtmp->r.min, display->black, ZP, mediumfont, buf);
    }
}
```

<function readnums 77a>≡ (92)

```
/* read one line of text from buffer and process integers */
int
readnums(Machine *m, int n, uulong *a, int spanlines)
{
    int i;
    char *p, *q, *ep;

    if(spanlines)
        ep = m->ebufp;
    else
        for(ep=m->bufp; ep<m->ebufp; ep++)
            if(*ep == '\n')
                break;
    p = m->bufp;
    for(i=0; i<n && p<ep; i++){
        while(p<ep && (!isascii(*p) || !isdigit(*p)) && *p!='-')
            p++;
        if(p == ep)
            break;
        a[i] = strtoull(p, &q, 10);
        p = q;
    }
    if(ep < m->ebufp)
        ep++;
    m->bufp = ep;
    return i == n;
}
```

<function filter 77b>≡ (92)

```
/* Network on fd1, mount driver on fd0 */
static int
filter(int fd)
{
    int p[2];

    if(pipe(p) < 0){
        fprintf(2, "stats: can't pipe: %r\n");
        killall("pipe");
    }

    switch(rfork(RFNOWAIT|RFPROC|RFFDG)) {
    case -1:
        sysfatal("rfork record module");
    case 0:
        dup(fd, 1);
        close(fd);
        dup(p[0], 0);
        close(p[0]);
        close(p[1]);
        execl("/bin/aux/fcall", "fcall", nil);
        fprintf(2, "stats: can't exec fcall: %r\n");
        killall("fcall");
    default:
        close(fd);
        close(p[0]);
    }
    return p[1];
}
```

<function connect9fs 78a>≡ (92)

```
/*
 * 9fs
 */
int
connect9fs(char *addr)
{
    char dir[256], *na;
    int fd;

    fprintf(2, "connect9fs...");
    na = netmkaddr(addr, 0, "9fs");

    fprintf(2, "dial %s...", na);
    if((fd = dial(na, 0, dir, 0)) < 0)
        return -1;

    fprintf(2, "dir %s...", dir);
    // if(strstr(dir, "tcp"))
    // fd = filter(fd);
    return fd;
}
```

<function old9p 78b>≡ (92)

```
int
old9p(int fd)
{
    int p[2];

    if(pipe(p) < 0)
        return -1;

    switch(rfork(RFPROC|RFFDG|RFNAMEG)) {
    case -1:
        return -1;
    case 0:
        if(fd != 1){
            dup(fd, 1);
            close(fd);
        }
        if(p[0] != 0){
            dup(p[0], 0);
            close(p[0]);
        }
        close(p[1]);
        if(0){
            fd = open("/sys/log/cpu", OWRITE);
            if(fd != 2){
                dup(fd, 2);
                close(fd);
            }
            execl("/bin/srvold9p", "srvold9p", "-ds", nil);
        } else
            execl("/bin/srvold9p", "srvold9p", "-s", nil);
        return -1;
    default:
        close(fd);
        close(p[0]);
    }
    return p[1];
}
```

```
}
```

<function connectexportfs 79a>≡ (92)

```
/*
 * exportfs
 */
int
connectexportfs(char *addr)
{
    char buf[ERRMAX], dir[256], *na;
    int fd, n;
    char *tree;
    AuthInfo *ai;

    tree = "/";
    na = netmkaddr(addr, 0, "exportfs");
    if((fd = dial(na, 0, dir, 0)) < 0)
        return -1;

    ai = auth_proxy(fd, auth_getkey, "proto=p9any role=client");
    if(ai == nil)
        return -1;

    n = write(fd, tree, strlen(tree));
    if(n < 0){
        close(fd);
        return -1;
    }

    strcpy(buf, "can't read tree");
    n = read(fd, buf, sizeof buf - 1);
    if(n!=2 || buf[0]!='0' || buf[1]!='K'){
        buf[sizeof buf - 1] = '\0';
        werrstr("bad remote tree: %s\n", buf);
        close(fd);
        return -1;
    }

    // if(strstr(dir, "tcp"))
    // fd = filter(fd);

    if(oldsystem)
        return old9p(fd);

    return fd;
}
```

<function readswap 79b>≡ (92)

```
int
readswap(Machine *m, uulong *a)
{
    if(strstr(m->buf, "memory\n")){
        /* new /dev/swap - skip first 3 numbers */
        if(!readnums(m, 7, a, 1))
            return 0;
        a[0] = a[3];
        a[1] = a[4];
        a[2] = a[5];
        a[3] = a[6];
        return 1;
    }
}
```



```

    }
    return readnums(m, nelem(m->devswap), a, 0);
}

⟨function shortname 80a⟩≡ (92)
char*
shortname(char *s)
{
    char *p, *e;

    p = estrdup(s);
    e = strchr(p, '.');
    if(e)
        *e = 0;
    return p;
}

⟨function ilog10 80b⟩≡ (92)
int
ilog10(uvlong j)
{
    int i;

    for(i = 0; j >= 10; i++)
        j /= 10;
    return i;
}

⟨function initmach 80c⟩≡ (92)
int
initmach(Machine *m, char *name)
{
    int n, fd;
    uvlong a[MAXNUM];
    char *p, mpt[256], buf[256];

    p = strchr(name, '!');
    if(p)
        p++;
    else
        p = name;
    m->name = estrdup(p);
    m->shortname = shortname(p);
    m->remote = (strcmp(p, mysysname) != 0);
    if(m->remote == 0)
        strcpy(mpt, "");
    else{
        snprintf(mpt, sizeof mpt, "/n/%s", p);
        fd = connectexportfs(name);
        if(fd < 0){
            fprintf(2, "can't connect to %s: %r\n", name);
            return 0;
        }
        /* BUG? need to use amount() now? */
        if(mount(fd, -1, mpt, MREPL, "") < 0){
            fprintf(2, "stats: mount %s on %s failed (%r); trying /n/sid\n", name, mpt);
            strcpy(mpt, "/n/sid");
            if(mount(fd, -1, mpt, MREPL, "") < 0){
                fprintf(2, "stats: mount %s on %s failed: %r\n", name, mpt);
                return 0;
            }
        }
    }
}

```

```

    }
}

snprintf(buf, sizeof buf, "%s/dev/swap", mpt);
m->swapfd = open(buf, OREAD);
if(loadbuf(m, &m->swapfd) && readswap(m, a))
    memmove(m->devswap, a, sizeof m->devswap);
else{
    m->devswap[Maxswap] = 100;
    m->devswap[Maxmem] = 100;
}

snprintf(buf, sizeof buf, "%s/dev/sysstat", mpt);
m->statsfd = open(buf, OREAD);
if(loadbuf(m, &m->statsfd)){
    for(n=0; readnums(m, nelem(m->devsysstat), a, 0); n++)
        ;
    m->nproc = n;
}else
    m->nproc = 1;
m->lgproc = ilog10(m->nproc);

snprintf(buf, sizeof buf, "%s/net/ether0/stats", mpt);
m->etherfd = open(buf, OREAD);
if(loadbuf(m, &m->etherfd) && readnums(m, nelem(m->netetherstats), a, 1))
    memmove(m->netetherstats, a, sizeof m->netetherstats);

snprintf(buf, sizeof buf, "%s/net/ether0/ifstats", mpt);
m->ifstatsfd = open(buf, OREAD);
if(loadbuf(m, &m->ifstatsfd)){
    /* need to check that this is a wavelan interface */
    if(strncmp(m->buf, "Signal: ", 8) == 0 && readnums(m, nelem(m->netetherifstats), a, 1))
        memmove(m->netetherifstats, a, sizeof m->netetherifstats);
}

snprintf(buf, sizeof buf, "%s/mnt/apm/battery", mpt);
m->batteryfd = open(buf, OREAD);
m->bitsybatfd = -1;
if(m->batteryfd >= 0){
    if(loadbuf(m, &m->batteryfd) && readnums(m, nelem(m->batterystats), a, 0))
        memmove(m->batterystats, a, sizeof(m->batterystats));
}else{
    snprintf(buf, sizeof buf, "%s/dev/battery", mpt);
    m->bitsybatfd = open(buf, OREAD);
    if(loadbuf(m, &m->bitsybatfd) && readnums(m, 1, a, 0))
        memmove(m->batterystats, a, sizeof(m->batterystats));
}
snprintf(buf, sizeof buf, "%s/dev/cputemp", mpt);
m->tempfd = open(buf, OREAD);
if(loadbuf(m, &m->tempfd))
    for(n=0; n < nelem(m->temp) && readnums(m, 2, a, 0); n++)
        m->temp[n] = a[0];
return 1;
}

```

$\langle global \text{ catchalarm } 81a \rangle \equiv$ (92)
 jmp_buf catchalarm;

$\langle function \text{ alarmed } 81b \rangle \equiv$ (92)

```

void
alarmed(void *a, char *s)
{
    if(strcmp(s, "alarm") == 0)
        notejmp(a, catchalarm, 1);
    noted(NDFLT);
}

⟨function needswap 82a⟩≡ (92)
int
needswap(int init)
{
    return init | present[Mmem] | present[Mswap];
}

⟨function needstat 82b⟩≡ (92)
int
needstat(int init)
{
    return init | present[Mcontext] | present[Mfault] | present[Mintr] | present[Mload] | present[Midle] |
        present[Minintr] | present[Msyscall] | present[Mtlbmiss] | present[Mtlbpurge];
}

⟨function needether 82c⟩≡ (92)
int
needether(int init)
{
    return init | present[Mether] | present[Metherin] | present[Metherout] | present[Methererr];
}

⟨function needbattery 82d⟩≡ (92)
int
needbattery(int init)
{
    return init | present[Mbattery];
}

⟨function needsignal 82e⟩≡ (92)
int
needsignal(int init)
{
    return init | present[Msignal];
}

⟨function needtemp 82f⟩≡ (92)
int
needtemp(int init)
{
    return init | present[Mtemp];
}

⟨function readmach 82g⟩≡ (92)
void
readmach(Machine *m, int init)
{
    int n, i;
    uulong a[nelem(m->devsysstat)];
    char buf[32];

    if(m->remote && (m->disable || setjmp(catchalarm))){

```

```

    if (m->disable++ >= 5)
        m->disable = 0; /* give it another chance */
    memmove(m->devsysstat, m->prevsysstat, sizeof m->devsysstat);
    memmove(m->netetherstats, m->prevetherstats, sizeof m->netetherstats);
    return;
}
snprint(buf, sizeof buf, "%s", m->name);
if (strcmp(m->name, buf) != 0){
    free(m->name);
    m->name = estrdup(buf);
    free(m->shortname);
    m->shortname = shortname(buf);
    if(display != nil) /* else we're still initializing */
        erezized(0);
}
if(m->remote){
    notify(alarmed);
    alarm(5000);
}
if(needswap(init) && loadbuf(m, &m->swapfd) && readswap(m, a))
    memmove(m->devswap, a, sizeof m->devswap);
if(needstat(init) && loadbuf(m, &m->statsfd)){
    memmove(m->prevsysstat, m->devsysstat, sizeof m->devsysstat);
    memset(m->devsysstat, 0, sizeof m->devsysstat);
    for(n=0; n<m->nproc && readnums(m, nelem(m->devsysstat), a, 0); n++)
        for(i=0; i<nelem(m->devsysstat); i++)
            m->devsysstat[i] += a[i];
}
if(needether(init) && loadbuf(m, &m->etherfd) && readnums(m, nelem(m->netetherstats), a, 1)){
    memmove(m->prevetherstats, m->netetherstats, sizeof m->netetherstats);
    memmove(m->netetherstats, a, sizeof m->netetherstats);
}
if(needsignal(init) && loadbuf(m, &m->ifstatsfd) && strncmp(m->buf, "Signal: ", 8)==0 && readnums(m, nelem(
    memmove(m->netetherifstats, a, sizeof m->netetherifstats);
}
if(needbattery(init) && loadbuf(m, &m->batteryfd) && readnums(m, nelem(m->batterystats), a, 0))
    memmove(m->batterystats, a, sizeof(m->batterystats));
if(needbattery(init) && loadbuf(m, &m->bitsybatfd) && readnums(m, 1, a, 0))
    memmove(m->batterystats, a, sizeof(m->batterystats));
if(needtemp(init) && loadbuf(m, &m->tempfd))
    for(n=0; n < nelem(m->temp) && readnums(m, 2, a, 0); n++)
        m->temp[n] = a[0];
if(m->remote){
    alarm(0);
    notify(nil);
}
}

```

⟨function memval 83a⟩≡ (92)

```

void
memval(Machine *m, uulong *v, uulong *vmax, int)
{
    *v = m->devswap[Mem];
    *vmax = m->devswap[Maxmem];
}

```

⟨function swapval 83b⟩≡ (92)

```

void
swapval(Machine *m, uulong *v, uulong *vmax, int)
{

```

```

    *v = m->devswap[Swap];
    *vmax = m->devswap[Maxswap];
}

⟨function contextval 84a⟩≡ (92)
void
contextval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->devsysstat[Context]-m->prevsysstat[Context];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}

⟨function intrval 84b⟩≡ (92)
/*
 * bug: need to factor in HZ
 */
void
intrval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->devsysstat[Interrupt]-m->prevsysstat[Interrupt];
    *vmax = sleeptime*m->nproc*10;
    if(init)
        *vmax = sleeptime*10;
}

⟨function syscallval 84c⟩≡ (92)
void
syscallval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->devsysstat[Syscall]-m->prevsysstat[Syscall];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}

⟨function faultval 84d⟩≡ (92)
void
faultval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->devsysstat[Fault]-m->prevsysstat[Fault];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}

⟨function tlbmissval 84e⟩≡ (92)
void
tlbmissval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->devsysstat[TLBfault]-m->prevsysstat[TLBfault];
    *vmax = (sleeptime/1000)*10*m->nproc;
    if(init)
        *vmax = (sleeptime/1000)*10;
}

```

```

⟨function tlbpurgeval 85a⟩≡ (92)
void
tlbpurgeval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->devsysstat[TLBpurge]-m->prevsysstat[TLBpurge];
    *vmax = (sleeptime/1000)*10*m->nproc;
    if(init)
        *vmax = (sleeptime/1000)*10;
}

⟨function loadval 85b⟩≡ (92)
void
loadval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->devsysstat[Load];
    *vmax = 1000*m->nproc;
    if(init)
        *vmax = 1000;
}

⟨function idleva 85c⟩≡ (92)
void
idleva(Machine *m, uulong *v, uulong *vmax, int)
{
    *v = m->devsysstat[Idle]/m->nproc;
    *vmax = 100;
}

⟨function inintrval 85d⟩≡ (92)
void
inintrval(Machine *m, uulong *v, uulong *vmax, int)
{
    *v = m->devsysstat[InIntr]/m->nproc;
    *vmax = 100;
}

⟨function etherval 85e⟩≡ (92)
void
etherval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->netetherstats[In]-m->prevetherstats[In] + m->netetherstats[Out]-m->prevetherstats[Out];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}

⟨function etherinval 85f⟩≡ (92)
void
etherinval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->netetherstats[In]-m->prevetherstats[In];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}

```

<function etheroutval 86a>≡ (92)

```
void
etheroutval(Machine *m, uulong *v, uulong *vmax, int init)
{
    *v = m->netetherstats[Out]-m->prevetherstats[Out];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}
```

<function ethererrval 86b>≡ (92)

```
void
ethererrval(Machine *m, uulong *v, uulong *vmax, int init)
{
    int i;

    *v = 0;
    for(i=Err0; i<nelem(m->netetherstats); i++)
        *v += m->netetherstats[i];
    *vmax = (sleeptime/1000)*10*m->nproc;
    if(init)
        *vmax = (sleeptime/1000)*10;
}
```

<function batteryval 86c>≡ (92)

```
void
batteryval(Machine *m, uulong *v, uulong *vmax, int)
{
    *v = m->batterystats[0];
    if(m->bitsybatfd >= 0)
        *vmax = 184; // at least on my bitsy...
    else
        *vmax = 100;
}
```

<function signalval 86d>≡ (92)

```
void
signalval(Machine *m, uulong *v, uulong *vmax, int)
{
    uulong l;

    *vmax = sleeptime;
    l = m->netetherifstats[0];
    /*
     * Range is seen to be from about -45 (strong) to -95 (weak); rescale
     */
    if(l == 0){ /* probably not present */
        *v = 0;
        return;
    }
    *v = 20*(l+95);
}
```

<function tempval 86e>≡ (92)

```
void
tempval(Machine *m, uulong *v, uulong *vmax, int)
{
    uulong l;

    *vmax = sleeptime;
```

```

    l = m->temp[0];
    if(l == ~0 || l == 0)
        *v = 0;
    else
        *v = (l-20)*27;
}

```

<function addgraph 87a>≡ (92)

```

void
addgraph(int n)
{
    Graph *g, *ograph;
    int i, j;
    static int nadd;

    if(n > nelem(menu2str))
        abort();
    /* avoid two adjacent graphs of same color */
    if(ngraph>0 && graph[ngraph-1].colindex==nadd%Ncolor)
        nadd++;
    ograph = graph;
    graph = emalloc(nmach*(ngraph+1)*sizeof(Graph));
    for(i=0; i<nmach; i++)
        for(j=0; j<ngraph; j++)
            graph[i*(ngraph+1)+j] = ograph[i*ngraph+j];
    free(oggraph);
    ngraph++;
    for(i=0; i<nmach; i++){
        g = &graph[i*ngraph+(ngraph-1)];
        memset(g, 0, sizeof(Graph));
        g->label = menu2str[n]+0pwid;
        g->newvalue = newvaluefn[n];
        g->update = update1; /* no other update functions yet */
        g->mach = &mach[i];
        g->colindex = nadd%Ncolor;
    }
    present[n] = 1;
    nadd++;
}

```

<function dropgraph 87b>≡ (92)

```

void
dropgraph(int which)
{
    Graph *oggraph;
    int i, j, n;

    if(which > nelem(menu2str))
        abort();
    /* convert n to index in graph table */
    n = -1;
    for(i=0; i<ngraph; i++)
        if(strcmp(menu2str[which]+0pwid, graph[i].label) == 0){
            n = i;
            break;
        }
    if(n < 0){
        fprintf(2, "stats: internal error can't drop graph\n");
        killall("error");
    }
}

```



```

ograph = graph;
graph = emalloc(nmach*(ngraph-1)*sizeof(Graph));
for(i=0; i<nmach; i++){
    for(j=0; j<n; j++){
        graph[i*(ngraph-1)+j] = ograph[i*ngraph+j];
        free(ograph[i*ngraph+j].data);
        freeimage(ograph[i*ngraph+j].overtmp);
        for(j++; j<ngraph; j++){
            graph[i*(ngraph-1)+j-1] = ograph[i*ngraph+j];
        }
    }
    free(ograph);
    ngraph--;
    present[which] = 0;
}

```

<function addmachine 88a>≡ (92)

```

int
addmachine(char *name)
{
    if(ngraph > 0){
        fprintf(2, "stats: internal error: ngraph>0 in addmachine()\n");
        usage();
    }
    if(mach == nil)
        nmach = 0; /* a little dance to get us started with local machine by default */
    mach = erealloc(mach, (nmach+1)*sizeof(Machine));
    memset(mach+nmach, 0, sizeof(Machine));
    if (initmach(mach+nmach, name)){
        nmach++;
        return 1;
    } else
        return 0;
}

```

<function labelstrs 88b>≡ (92)

```

void
labelstrs(Graph *g, char strs[Nlab][Lablen], int *np)
{
    int j;
    uvlong v, vmax;

    g->newvalue(g->mach, &v, &vmax, 1);
    if(logscale){
        for(j=1; j<=2; j++){
            sprintf(strs[j-1], "%g", scale*pow(10., j)*(double)vmax/100.);
            *np = 2;
        }
    }else{
        for(j=1; j<=3; j++){
            sprintf(strs[j-1], "%g", scale*(double)j*(double)vmax/4.0);
            *np = 3;
        }
    }
}

```

<function labelwidth 88c>≡ (92)

```

int
labelwidth(void)
{
    int i, j, n, w, maxw;
    char strs[Nlab][Lablen];

```

```

maxw = 0;
for(i=0; i<ngraph; i++){
    /* choose value for rightmost graph */
    labelstrs(&graph[ngraph*(nmach-1)+i], strs, &n);
    for(j=0; j<n; j++){
        w = stringwidth(mediumfont, strs[j]);
        if(w > maxw)
            maxw = w;
    }
}
return maxw;
}

```

```

⟨function resize 89⟩≡ (92)
void
resize(void)
{
    int i, j, k, n, startx, starty, x, y, dx, dy, ly, ondata, maxx, wid, nlab;
    Graph *g;
    Rectangle machr, r;
    uvlong v, vmax;
    char buf[128], labs[Nlab][Lablen];

    draw(view, view->r, display->white, nil, ZP);

    /* label left edge */
    x = view->r.min.x;
    y = view->r.min.y + Labspace+mediumfont->height+Labspace;
    dy = (view->r.max.y - y)/ngraph;
    dx = Labspace+stringwidth(mediumfont, "0")+Labspace;
    startx = x+dx+1;
    starty = y;
    for(i=0; i<ngraph; i++,y+=dy){
        draw(view, Rect(x, y-1, view->r.max.x, y), display->black, nil, ZP);
        draw(view, Rect(x, y, x+dx, view->r.max.y), cols[graph[i].colindex][0], nil, paritypt(x));
        label(Pt(x, y), dy, graph[i].label);
        draw(view, Rect(x+dx, y, x+dx+1, view->r.max.y), cols[graph[i].colindex][2], nil, ZP);
    }

    /* label top edge */
    dx = (view->r.max.x - startx)/nmach;
    for(x=startx, i=0; i<nmach; i++,x+=dx){
        draw(view, Rect(x-1, starty-1, x, view->r.max.y), display->black, nil, ZP);
        j = dx/stringwidth(mediumfont, "0");
        n = mach[i].nproc;
        if(n>1 && j>=1+3+mach[i].lgproc){ /* first char of name + (n) */
            j -= 3+mach[i].lgproc;
            if(j <= 0)
                j = 1;
            snprintf(buf, sizeof buf, "%.s(%d)", j, mach[i].shortname, n);
        }else
            snprintf(buf, sizeof buf, "%.s", j, mach[i].shortname);
        string(view, Pt(x+Labspace, view->r.min.y + Labspace), display->black, ZP, mediumfont, buf);
    }

    maxx = view->r.max.x;

    /* label right, if requested */
    if(ylabels && dy>Nlab*(mediumfont->height+1)){
        wid = labelwidth();
    }
}

```

```

if(wid < (maxx-startx)-30){
    /* else there's not enough room */
    maxx -= 1+Lx+wid;
    draw(view, Rect(maxx, starty, maxx+1, view->r.max.y), display->black, nil, ZP);
    y = starty;
    for(j=0; j<ngraph; j++, y+=dy){
        /* choose value for rightmost graph */
        g = &graph[ngraph*(nmach-1)+j];
        labelstrs(g, labs, &nlab);
        r = Rect(maxx+1, y, view->r.max.x, y+dy-1);
        if(j == ngraph-1)
            r.max.y = view->r.max.y;
        draw(view, r, cols[g->colindex][0], nil, paritypt(r.min.x));
        for(k=0; k<nlab; k++){
            ly = y + (dy*(nlab-k)/(nlab+1));
            draw(view, Rect(maxx+1, ly, maxx+1+Lx, ly+1), display->black, nil, ZP);
            ly -= mediumfont->height/2;
            string(view, Pt(maxx+1+Lx, ly), display->black, ZP, mediumfont, labs[k]);
        }
    }
}

/* create graphs */
for(i=0; i<nmach; i++){
    machr = Rect(startx+i*dx, starty, maxx, view->r.max.y);
    if(i < nmach-1)
        machr.max.x = startx+(i+1)*dx - 1;
    y = starty;
    for(j=0; j<ngraph; j++, y+=dy){
        g = &graph[i*ngraph+j];
        /* allocate data */
        ondata = g->ndata;
        g->ndata = Dx(machr)+1; /* may be too many if label will be drawn here; so what? */
        g->data = erealloc(g->data, g->ndata*sizeof(g->data[0]));
        if(g->ndata > ondata)
            memset(g->data+ondata, 0, (g->ndata-ondata)*sizeof(g->data[0]));
        /* set geometry */
        g->r = machr;
        g->r.min.y = y;
        g->r.max.y = y+dy - 1;
        if(j == ngraph-1)
            g->r.max.y = view->r.max.y;
        draw(view, g->r, cols[g->colindex][0], nil, paritypt(g->r.min.x));
        g->overflow = 0;
        r = g->r;
        r.max.y = r.min.y+mediumfont->height;
        r.max.x = r.min.x+stringwidth(mediumfont, "999999999999");
        freeimage(g->overtmp);
        g->overtmp = nil;
        if(r.max.x <= g->r.max.x)
            g->overtmp = allocimage(display, r, view->chan, 0, -1);
        g->newvalue(g->mach, &v, &vmax, 0);
        redraw(g, vmax);
    }
}

flushimage(display, 1);
}

```

```

⟨function eresized 91a)≡ (92)
void
eresized(int new)
{
    lockdisplay(display);
    if(new && getwindow(display, Refnone) < 0) {
        fprintf(2, "stats: can't reattach to window\n");
        killall("reattach");
    }
    resize();
    unlockdisplay(display);
}

```

```

⟨function mouseproc 91b)≡ (92)
void
mouseproc(void)
{
    Mouse mouse;
    int i;

    for(;;){
        mouse = emouse();
        if(mouse.buttons == 4){
            lockdisplay(display);
            for(i=0; i<Nmenu2; i++)
                if(present[i])
                    memmove(menu2str[i], "drop ", 0pwid);
                else
                    memmove(menu2str[i], "add  ", 0pwid);
            i = emenuhit(3, &mouse, &menu2);
            if(i >= 0){
                if(!present[i])
                    addgraph(i);
                else if(ngraph > 1)
                    dropgraph(i);
                resize();
            }
            unlockdisplay(display);
        }
    }
}

```

```

⟨function startproc 91c)≡ (92)
void
startproc(void (*f)(void), int index)
{
    int pid;

    switch(pid = rfork(RFPROC|RFMEM|RFNOWAIT)){
    case -1:
        fprintf(2, "stats: fork failed: %r\n");
        killall("fork failed");
    case 0:
        f();
        fprintf(2, "stats: %s process exits\n", procnames[index]);
        if(index >= 0)
            killall("process died");
        exits(nil);
    }
    if(index >= 0)

```

```

        pids[index] = pid;
    }

<misc/stats.c 92>≡
#include <u.h>
#include <libc.h>
#include <ctype.h>
#include <auth.h>
#include <fcall.h>
#include <draw.h>
#include <window.h>
#include <event.h>

<constant MAXNUM 58a>

typedef struct Graph Graph;
typedef struct Machine Machine;

<struct Graph 58b>

<enum _anon_ (misc/stats.c) 58c>

<struct Machine 59a>

<enum _anon_ (misc/stats.c)2 59b>

<enum _anon_ (misc/stats.c)3 59c>

<enum Menu2 60a>

<global menu2str 60b>

void contextval(Machine*, uulong*, uulong*, int),
    etherval(Machine*, uulong*, uulong*, int),
    ethererrval(Machine*, uulong*, uulong*, int),
    etherinval(Machine*, uulong*, uulong*, int),
    etheroutval(Machine*, uulong*, uulong*, int),
    faultval(Machine*, uulong*, uulong*, int),
    intrval(Machine*, uulong*, uulong*, int),
    inintrval(Machine*, uulong*, uulong*, int),
    loadval(Machine*, uulong*, uulong*, int),
    idleval(Machine*, uulong*, uulong*, int),
    memval(Machine*, uulong*, uulong*, int),
    swapval(Machine*, uulong*, uulong*, int),
    syscallval(Machine*, uulong*, uulong*, int),
    tlbmissval(Machine*, uulong*, uulong*, int),
    tlbpurgeval(Machine*, uulong*, uulong*, int),
    batteryval(Machine*, uulong*, uulong*, int),
    signalval(Machine*, uulong*, uulong*, int),
    tempval(Machine*, uulong*, uulong*, int);

<global menu2 60c>
<global present 60d>
<global newvaluefn 60e>

<global cols (misc/stats.c) 61a>
<global graph 61b>
<global mach 61c>
<global mediumfont 61d>

```

<global mysysname 61e>
<global argchars 61f>
<global pids 61g>
<global parity 61h>
<global nmach 61i>
<global ngraph 61j>
<global scale 61k>
<global logscale 61l>
<global ylabels 61m>
<global oldsystem 61n>
<global sleeptime 61o>

<global procnames 62a>

<function killall 73a>

<function emalloc 73b>

<function erealloc 73c>

<function estrdup 73d>

<function mkcol (misc/stats.c) 74a>

<function colinit (misc/stats.c) 74b>

<function loadbuf 74c>

<function label 74d>

<function paritypt 75a>

<function datapoint 75b>

<function drawdatum 76a>

<function redraw (misc/stats.c) 76b>

<function update1 76c>

<function readnums 77a>

<function filter 77b>

<function connect9fs 78a>

<function old9p 78b>

<function connectexportfs 79a>

<function readswap 79b>

<function shortname 80a>

<function ilog10 80b>

<function initmach 80c>

<global catchalarm 81a>

<function alarmed 81b>

<function needswap 82a>

<function needstat 82b>

<function needether 82c>

<function needbattery 82d>

<function needsignal 82e>

<function needtemp 82f>

<function readmach 82g>

<function memval 83a>

<function swapval 83b>

<function contextval 84a>

<function intrval 84b>

<function syscallval 84c>

<function faultval 84d>

<function tlbmissval 84e>

<function tlbpurgeval 85a>

<function loadval 85b>

<function idleval 85c>

<function inintrval 85d>

<function etherval 85e>

<function etherinval 85f>

<function etheroutval 86a>

<function ethererrval 86b>

<function batteryval 86c>

<function signalval 86d>

<function tempval 86e>

<function usage (misc/stats.c) 11c>

<function addgraph 87a>

<function dropgraph 87b>

<function addmachine 88a>
 <function labelstrs 88b>
 <function labelwidth 88c>
 <function resize 89>
 <function eresized 91a>
 <function mouseproc 91b>
 <function startproc 91c>
 <function main (misc/stats.c) 62b>

A.2 iostats/

A.2.1 iostats/statfs.h

```

<constant DEBUGFILE 95a>≡ (97c)
/*
 * statfs.h - definitions for statistic gathering file server
 */

#define DEBUGFILE "iostats.out"

<constant DONESTR 95b>≡ (97c)
#define DONESTR "done"

<constant DEBUG 95c>≡ (97c)
#define DEBUG if(dbg)fprint

<constant MAXPROC 95d>≡ (97c)
#define MAXPROC 16

<constant FHASHSIZE 95e>≡ (97c)
#define FHASHSIZE 64

<function fidhash 95f>≡ (97c)
#define fidhash(s) fhash[s%FHASHSIZE]

<enum _anon_ (iostats/statfs.h) 95g>≡ (97c)
enum{
    Maxfdata = 8192, /* max size of data in 9P message */
    Maxrpc = 20000, /* number of RPCs we'll log */
};

<struct Frec 95h>≡ (97c)
struct Frec
{
    Frec *next;
    char *op;
    ulong nread;
    ulong nwrite;
    ulong bread;
    ulong bwrite;
    ulong opens;
};
  
```


<struct Rpc 96a>≡ (97c)

```
struct Rpc
{
    char *name;
    ulong count;
    vlong time;
    vlong lo;
    vlong hi;
    ulong bin;
    ulong bout;
};
```

<struct Stats 96b>≡ (97c)

```
struct Stats
{
    ulong toread;
    ulong totwrite;
    ulong nrpc;
    ulong nproto;
    Rpc rpc[Maxrpc];
};
```

<struct Fsrpc 96c>≡ (97c)

```
struct Fsrpc
{
    int busy; /* Work buffer has pending rpc to service */
    uintptr pid; /* Pid of slave process executing the rpc */
    int canint; /* Interrupt gate */
    int flushtag; /* Tag on which to reply to flush */
    Fcall work; /* Plan 9 incoming Fcall */
    uchar buf[IOHDRSZ+Maxfdata]; /* Data buffer */
};
```

<struct Fid 96d>≡ (97c)

```
struct Fid
{
    int fid; /* system fd for i/o */
    File *f; /* File attached to this fid */
    int mode;
    int nr; /* fid number */
    Fid *next; /* hash link */
    ulong nread;
    ulong nwrite;
    ulong bread;
    ulong bwrite;
    vlong offset; /* for directories */
};
```

<struct File 96e>≡ (97c)

```
struct File
{
    char *name;
    Qid qid;
    int inval;
    File *parent;
    File *child;
    File *childlist;
};
```

<struct Proc 97a>≡ (97c)

```
struct Proc
{
    uintptr pid;
    int busy;
    Proc *next;
};
```

<enum _anon_ (iostats/statfs.h)2 97b>≡ (97c)

```
enum
{
    Nr_workbufs = 40,
    Dseepad = 8192,
    Fidchunk = 1000,
};
```

<iostats/statfs.h 97c>≡

```
<constant DEBUGFILE 95a>
<constant DONESTR 95b>
<constant DEBUG 95c>
<constant MAXPROC 95d>
<constant FHASHSIZE 95e>
<function fidhash 95f>
```

<enum _anon_ (iostats/statfs.h) 95g>

```
typedef struct Fsrpc Fsrpc;
typedef struct Fid Fid;
typedef struct File File;
typedef struct Proc Proc;
typedef struct Stats Stats;
typedef struct Rpc Rpc;
typedef struct Frec Frec;
```

<struct Frec 95h>

<struct Rpc 96a>

<struct Stats 96b>

<struct Fsrpc 96c>

<struct Fid 96d>

<struct File 96e>

<struct Proc 97a>

<enum _anon_ (iostats/statfs.h)2 97b>

```
extern Fsrpc *Workq;
extern int dbg;
extern File *root;
extern Fid **fhash;
extern Fid *fidfree;
extern int qid;
extern Proc *Proclist;
extern int done;
extern Stats *stats;
extern Frec *frhead;
```

```

extern Frec *frrtail;
extern int myiounit;

/* File system protocol service procedures */
void Xcreate(Fsrpc*), Xclunk(Fsrpc*);
void Xversion(Fsrpc*), Xauth(Fsrpc*), Xflush(Fsrpc*);
void Xattach(Fsrpc*), Xwalk(Fsrpc*), Xauth(Fsrpc*);
void Xremove(Fsrpc*), Xstat(Fsrpc*), Xwstat(Fsrpc*);
void slave(Fsrpc*);

void reply(Fcall*, Fcall*, char*);
Fid *getfid(int);
int freefid(int);
Fid *newfid(int);
Fsrpc *getsbuf(void);
void initroot(void);
void fatal(char*);
void makepath(char*, File*, char*);
File *file(File*, char*);
void slaveopen(Fsrpc*);
void slaveread(Fsrpc*);
void slavewrite(Fsrpc*);
void blockingslave(void);
void reopen(Fid *f);
void noteproc(int, char*);
void flushaction(void*, char*);
void catcher(void*, char*);
ulong msec(void);
void fidreport(Fid*);

```

A.2.2 iostats/globals.c

<i><global Workq 98a></i> ≡	(99d)
Fsrpc *Workq;	
<i><global dbg 98b></i> ≡	(99d)
int dbg;	
<i><global root 98c></i> ≡	(99d)
File *root;	
<i><global fhash 98d></i> ≡	(99d)
Fid **fhash;	
<i><global fidfree 98e></i> ≡	(99d)
Fid *fidfree;	
<i><global qid 98f></i> ≡	(99d)
int qid;	
<i><global Procllist 98g></i> ≡	(99d)
Proc *Procllist;	
<i><global done 98h></i> ≡	(99d)
int done;	
<i><global stats 98i></i> ≡	(99d)
Stats *stats;	

<global frhead 99a>≡ (99d)

```
Frec *frhead;
```

<global frtail 99b>≡ (99d)

```
Frec *frtail;
```

<global myiounit 99c>≡ (99d)

```
int myiounit;
```

<iostats/globals.c 99d>≡

```
#include <u.h>
#include <libc.h>
#include <auth.h>
#include <fcall.h>

#include "statfs.h"
```

```
<global Workq 98a>
<global dbg 98b>
<global root 98c>
<global fhash 98d>
<global fidfree 98e>
<global qid 98f>
<global Proclist 98g>
<global done 98h>
<global stats 98i>
<global frhead 99a>
<global frtail 99b>
<global myiounit 99c>
```

A.2.3 iostats/iostats.c

<global fcalls 99e>≡ (109)

```
void (*fcalls[])(Fsrpc*) =
{
    [Tversion] Xversion,
    [Tauth] Xauth,
    [Tflush] Xflush,
    [Tattach] Xattach,
    [Twalk] Xwalk,
    [Topen] slave,
    [Tcreate] Xcreate,
    [Tclunk] Xclunk,
    [Tread] slave,
    [Twrite] slave,
    [Tremove] Xremove,
    [Tstat] Xstat,
    [Twstat] Xwstat,
};
```

<global p 99f>≡ (109)

```
int p[2];
```

<function usage (iostats/iostats.c) 99g>≡ (109)

```
static void
usage(void)
{
    fprintf(2, "usage: iostats [-d] [-f debugfile] cmds [args ...]\n");
    exits("usage");
}
```

<function main (iostats/iostats.c) 100>≡ (109)

```
void
main(int argc, char **argv)
{
    Fsrpc *r;
    Rpc *rpc;
    Proc *m;
    Frec *fr;
    Fid *fid;
    ulong ttime;
    char *dbfile, *s;
    char buf[128];
    float brpsec, bwpsec, bppsec;
    int type, cpid, fspid, n;

    dbfile = DEBUGFILE;

    ARGBEGIN{
    case 'd':
        dbg++;
        break;
    case 'f':
        dbfile = ARGF();
        break;
    default:
        usage();
    }ARGEND

    if(argc == 0)
        usage();

    if(dbg) {
        close(2);
        create(dbfile, OWRITE, 0666);
    }

    if(pipe(p) < 0)
        fatal("pipe");

    switch(cpid = fork()) {
    case -1:
        fatal("fork");
    case 0:
        close(p[1]);
        if(getwd(buf, sizeof(buf)) == 0)
            fatal("no working directory");

        rfork(RFENVG|RFNAMEG|RFNOTEG);
        if(mount(p[0], -1, "/", MREPL, "") < 0)
            fatal("mount /");

        bind("#c/pid", "/dev/pid", MREPL);
        bind("#e", "/env", MREPL|MCREATE);
        close(0);
        close(1);
        close(2);
        open("/fd/0", OREAD);
        open("/fd/1", OWRITE);
        open("/fd/2", OWRITE);
```

```

    if(chdir(buf) < 0)
        fatal("chdir");

    runprog(argv);
default:
    close(p[0]);
}

switch(fspid = fork()) {
default:
    while(cpid != waitpid())
        ;
    postnote(PNPROC, fspid, DONESTR);
    while(fspid != waitpid())
        ;
    exits(0);
case -1:
    fatal("fork");
case 0:
    break;
}

/* Allocate work queues in shared memory */
malloc(Dseghpad);
Workq = malloc(sizeof(Fsrpc)*Nr_workbufs);
stats = malloc(sizeof(Stats));
fhash = mallocz(sizeof(Fid*)*FHASHSIZE, 1);

if(Workq == 0 || fhash == 0 || stats == 0)
    fatal("no initial memory");

memset(Workq, 0, sizeof(Fsrpc)*Nr_workbufs);
memset(stats, 0, sizeof(Stats));

stats->rpc[Tversion].name = "version";
stats->rpc[Tauth].name = "auth";
stats->rpc[Tflush].name = "flush";
stats->rpc[Tattach].name = "attach";
stats->rpc[Twalk].name = "walk";
stats->rpc[Topen].name = "open";
stats->rpc[Tcreate].name = "create";
stats->rpc[Tclunk].name = "clunk";
stats->rpc[Tread].name = "read";
stats->rpc[Twrite].name = "write";
stats->rpc[Tremove].name = "remove";
stats->rpc[Tstat].name = "stat";
stats->rpc[Twstat].name = "wstat";

for(n = 0; n < Maxrpc; n++)
    stats->rpc[n].lo = 10000000000LL;

fmtinstall('M', dirmodefmt);
fmtinstall('D', dirfmt);
fmtinstall('F', fcallfmt);

if(chdir("/") < 0)
    fatal("chdir");

initroot();

```

```

DEBUG(2, "statfs: %s\n", buf);

notify(catcher);

for(;;) {
    r = getsbuf();
    if(r == 0)
        fatal("Out of service buffers");

    n = read9pmsg(p[1], r->buf, sizeof(r->buf));
    if(done)
        break;
    if(n < 0)
        fatal("read server");

    if(convM2S(r->buf, n, &r->work) == 0)
        fatal("format error");

    stats->nrpc++;
    stats->nproto += n;

    DEBUG(2, "%F\n", &r->work);

    type = r->work.type;
    rpc = &stats->rpc[type];
    rpc->count++;
    rpc->bin += n;
    (fcalls[type])(r);
}

/* Clear away the slave children */
for(m = Proclist; m; m = m->next)
    postnote(PNPROC, m->pid, "kill");

rpc = &stats->rpc[Tread];
brpsec = (float)stats->totread / (((float)rpc->time/1e9)+.000001);

rpc = &stats->rpc[Twrite];
bwpsec = (float)stats->totwrite / (((float)rpc->time/1e9)+.000001);

ttime = 0;
for(n = 0; n < Maxrpc; n++) {
    rpc = &stats->rpc[n];
    if(rpc->count == 0)
        continue;
    ttime += rpc->time;
}

bppsec = (float)stats->nproto / ((ttime/1e9)+.000001);

fprintf(2, "\nread      %ld bytes, %g Kb/sec\n", stats->totread, brpsec/1024.0);
fprintf(2, "write       %ld bytes, %g Kb/sec\n", stats->totwrite, bwpsec/1024.0);
fprintf(2, "protocol   %ld bytes, %g Kb/sec\n", stats->nproto, bppsec/1024.0);
fprintf(2, "rpc        %ld count\n", stats->nrpc);

fprintf(2, "%-10s %5s %5s %5s %5s %5s      T      R\n",
        "Message", "Count", "Low", "High", "Time", "Averg");

for(n = 0; n < Maxrpc; n++) {
    rpc = &stats->rpc[n];

```

```

    if(rpc->count == 0)
        continue;
    fprintf(2, "%-10s %5lud %5llud %5llud %5llud %5llud ms %8lud %8lud bytes\n",
        rpc->name,
        rpc->count,
        rpc->lo/1000000,
        rpc->hi/1000000,
        rpc->time/1000000,
        rpc->time/1000000/rpc->count,
        rpc->bin,
        rpc->bout);
}

for(n = 0; n < FHASHSIZE; n++)
    for(fid = fhash[n]; fid; fid = fid->next)
        if(fid->nread || fid->nwrite)
            fidreport(fid);
if(frhead == 0)
    exits(0);

fprintf(2, "\nOpens    Reads  (bytes)    Writes  (bytes) File\n");
for(fr = frhead; fr; fr = fr->next) {
    s = fr->op;
    if(*s) {
        if(strcmp(s, "/fd/0") == 0)
            s = "(stdin)";
        else
            if(strcmp(s, "/fd/1") == 0)
                s = "(stdout)";
            else
                if(strcmp(s, "/fd/2") == 0)
                    s = "(stderr)";
    }
    else
        s = "/.";

    fprintf(2, "%5lud %8lud %8lud %8lud %8lud %s\n", fr->opens, fr->nread, fr->bread,
        fr->nwrite, fr->bwrite, s);
}

exits(0);
}

```

<function reply 103>≡ (109)

```

void
reply(Fcall *r, Fcall *t, char *err)
{
    uchar data[IOHDRSZ+Maxfdata];
    int n;

    t->tag = r->tag;
    t->fid = r->fid;
    if(err) {
        t->type = Rerror;
        t->ename = err;
    }
    else
        t->type = r->type + 1;

    DEBUG(2, "\t%F\n", t);
}

```



```

    n = convS2M(t, data, sizeof data);
    if(write(p[1], data, n)!=n)
        fatal("mount write");
    stats->nproto += n;
    stats->rpc[t->type-1].bout += n;
}

```

<function getfid 104a>≡ (109)

```

Fid *
getfid(int nr)
{
    Fid *f;

    for(f = fidhash(nr); f; f = f->next)
        if(f->nr == nr)
            return f;

    return 0;
}

```

<function freefid 104b>≡ (109)

```

int
freefid(int nr)
{
    Fid *f, **l;

    l = &fidhash(nr);
    for(f = *l; f; f = f->next) {
        if(f->nr == nr) {
            *l = f->next;
            f->next = fidfree;
            fidfree = f;
            return 1;
        }
        l = &f->next;
    }

    return 0;
}

```

<function newfid 104c>≡ (109)

```

Fid *
newfid(int nr)
{
    Fid *new, **l;
    int i;

    l = &fidhash(nr);
    for(new = *l; new; new = new->next)
        if(new->nr == nr)
            return 0;

    if(fidfree == 0) {
        fidfree = mallocz(sizeof(Fid) * Fidchunk, 1);
        if(fidfree == 0)
            fatal("out of memory");

        for(i = 0; i < Fidchunk-1; i++)
            fidfree[i].next = &fidfree[i+1];
    }
}

```

```

        fidfree[Fidchunk-1].next = 0;
    }

    new = fidfree;
    fidfree = new->next;

    memset(new, 0, sizeof(Fid));
    new->next = *l;
    *l = new;
    new->nr = nr;
    new->fid = -1;
    new->nread = 0;
    new->nwrite = 0;
    new->bread = 0;
    new->bwrite = 0;

    return new;
}

```

<function getsbuf 105a>≡ (109)

```

Fsrpc *
getsbuf(void)
{
    static int ap;
    int look;
    Fsrpc *wb;

    for(look = 0; look < Nr_workbufs; look++) {
        if(++ap == Nr_workbufs)
            ap = 0;
        if(Workq[ap].busy == 0)
            break;
    }

    if(look == Nr_workbufs)
        fatal("No more work buffers");

    wb = &Workq[ap];
    wb->pid = 0;
    wb->canint = 0;
    wb->flushtag = NOTAG;
    wb->busy = 1;

    return wb;
}

```

<function strcatalloc 105b>≡ (109)

```

char *
strcatalloc(char *p, char *n)
{
    char *v;

    v = realloc(p, strlen(p)+strlen(n)+1);
    if(v == 0)
        fatal("no memory");
    strcat(v, n);
    return v;
}

```

<function file 106a>≡ (109)

```
File *
file(File *parent, char *name)
{
    char buf[128];
    File *f, *new;
    Dir *dir;

    DEBUG(2, "\tfile: 0x%p %s name %s\n", parent, parent->name, name);

    for(f = parent->child; f; f = f->childlist)
        if(strcmp(name, f->name) == 0)
            break;

    if(f != nil && !f->inval)
        return f;
    makepath(buf, parent, name);
    dir = dirstat(buf);
    if(dir == nil)
        return 0;
    if(f != nil){
        free(dir);
        f->inval = 0;
        return f;
    }

    new = malloc(sizeof(File));
    if(new == 0)
        fatal("no memory");

    memset(new, 0, sizeof(File));
    new->name = strdup(name);
    if(new->name == nil)
        fatal("can't strdup");
    new->qid.type = dir->qid.type;
    new->qid.vers = dir->qid.vers;
    new->qid.path = ++qid;

    new->parent = parent;
    new->childlist = parent->child;
    parent->child = new;

    free(dir);
    return new;
}
```

<function initroot 106b>≡ (109)

```
void
initroot(void)
{
    Dir *dir;

    root = malloc(sizeof(File));
    if(root == 0)
        fatal("no memory");

    memset(root, 0, sizeof(File));
    root->name = strdup("/");
    if(root->name == nil)
        fatal("can't strdup");
}
```

```

    dir = dirstat(root->name);
    if(dir == nil)
        fatal("root stat");

    root->qid.type = dir->qid.type;
    root->qid.vers = dir->qid.vers;
    root->qid.path = ++qid;
    free(dir);
}

```

<function makepath 107a>≡ (109)

```

void
makepath(char *as, File *p, char *name)
{
    char *c, *seg[100];
    int i;
    char *s;

    seg[0] = name;
    for(i = 1; i < 100 && p; i++, p = p->parent){
        seg[i] = p->name;
        if(strcmp(p->name, "/") == 0)
            seg[i] = ""; /* will insert slash later */
    }

    s = as;
    while(i--) {
        for(c = seg[i]; *c; c++)
            *s++ = *c;
        *s++ = '/';
    }
    while(s[-1] == '/')
        s--;
    *s = '\0';
    if(as == s) /* empty string is root */
        strcpy(as, "/");
}

```

<function fatal 107b>≡ (109)

```

void
fatal(char *s)
{
    Proc *m;

    fprintf(2, "iostats: %s: %r\n", s);

    /* Clear away the slave children */
    for(m = Proclist; m; m = m->next)
        postnote(PNPROC, m->pid, "exit");

    exits("fatal");
}

```

<function rdenv 107c>≡ (109)

```

char*
rdenv(char *v, char **end)
{
    int fd, n;
    char *buf;
    Dir *d;

```

```

if((fd = open(v, OREAD)) == -1)
    return nil;
d = dirfstat(fd);
if(d == nil || (buf = malloc(d->length + 1)) == nil)
    return nil;
n = (int)d->length;
n = read(fd, buf, n);
close(fd);
if(n <= 0){
    free(buf);
    buf = nil;
}else{
    if(buf[n-1] != '\0')
        buf[n++] = '\0';
    *end = &buf[n];
}
free(d);
return buf;
}

```

<global Defaultpath 108a>≡ (109)
char Defaultpath[] = ".\0/bin";

<function runprog 108b>≡ (109)
void
runprog(char *argv[])
{
 char *path, *ep, *p;
 char arg0[256];

 path = rdenv("/env/path", &ep);
 if(path == nil){
 path = Defaultpath;
 ep = path+sizeof(Defaultpath);
 }
 for(p = path; p < ep; p += strlen(p)+1){
 snprintf(arg0, sizeof arg0, "%s/%s", p, argv[0]);
 exec(arg0, argv);
 }
 fatal("exec");
}

<function catcher 108c>≡ (109)
void
catcher(void *a, char *msg)
{
 USED(a);
 if(strcmp(msg, DONESTR) == 0) {
 done = 1;
 noted(NCONT);
 }
 if(strcmp(msg, "exit") == 0)
 exits("exit");

 noted(NDFLT);
}

<function fidreport 108d>≡ (109)
void
fidreport(Fid *f)

```

{
    char *p, path[128];
    Frec *fr;

    p = path;
    makepath(p, f->f, "");

    for(fr = frhead; fr; fr = fr->next) {
        if(strcmp(fr->op, p) == 0) {
            fr->nread += f->nread;
            fr->nwrite += f->nwrite;
            fr->bread += f->bread;
            fr->bwrite += f->bwrite;
            fr->opens++;
            return;
        }
    }

    fr = malloc(sizeof(Frec));
    if(fr == 0 || (fr->op = strdup(p)) == 0)
        fatal("no memory");

    fr->nread = f->nread;
    fr->nwrite = f->nwrite;
    fr->bread = f->bread;
    fr->bwrite = f->bwrite;
    fr->opens = 1;
    if(frhead == 0) {
        frhead = fr;
        frtail = fr;
    }
    else {
        frtail->next = fr;
        frtail = fr;
    }
    fr->next = 0;
}

```

<iostats/iostats.c 109>≡

```

/*
 * iostats - Gather file system information
 */

```

```

#include <u.h>
#include <libc.h>
#include <auth.h>
#include <fcall.h>

```

```

#include "statfs.h"

```

```

void runprog(char**);

```

<global fcalls 99e>

<global p 99f>

<function usage (iostats/iostats.c) 99g>

<function main (iostats/iostats.c) 100>

<function reply 103>

<function getfid 104a>
 <function freefid 104b>
 <function newfid 104c>
 <function getsbuf 105a>
 <function strcatalloc 105b>
 <function file 106a>
 <function initroot 106b>
 <function makepath 107a>
 <function fatal 107b>
 <function rdenv 107c>
 <global Defaultpath 108a>
 <function runprog 108b>
 <function catcher 108c>
 <function fidreport 108d>

A.2.4 iostats/statsrv.c

<global Ebadfid 110a>≡ (122b)
 char Ebadfid[] = "Bad fid";
 <global Enotdir 110b>≡ (122b)
 char Enotdir[] = "Not a directory";
 <global Edupfid 110c>≡ (122b)
 char Edupfid[] = "Fid already in use";
 <global Eopen 110d>≡ (122b)
 char Eopen[] = "Fid already opened";
 <global Exmnt 110e>≡ (122b)
 char Exmnt[] = "Cannot .. past mount point";
 <global Enoauth 110f>≡ (122b)
 char Enoauth[] = "iostats: Authentication failed";
 <global Ebadver 110g>≡ (122b)
 char Ebadver[] = "Unrecognized 9P version";

```

⟨function okfile 111a⟩≡ (122b)
int
okfile(char *s, int mode)
{
    if(strncmp(s, "/fd/", 3) == 0){
        /* 0, 1, and 2 we handle ourselves */
        if(s[4]=='/' || atoi(s+4) > 2)
            return 0;
        return 1;
    }
    if(strncmp(s, "/net/ssl", 8) == 0)
        return 0;
    if(strncmp(s, "/net/tls", 8) == 0)
        return 0;
    if(strncmp(s, "/srv/", 5) == 0 && ((mode&3) == OWRITE || (mode&3) == ORDWR))
        return 0;
    return 1;
}

```

```

⟨function update 111b⟩≡ (122b)
void
update(Rpc *rpc, vlong t)
{
    vlong t2;

    t2 = nsec();
    t = t2 - t;
    if(t < 0)
        t = 0;

    rpc->time += t;
    if(t < rpc->lo)
        rpc->lo = t;
    if(t > rpc->hi)
        rpc->hi = t;
}

```

```

⟨function Xversion 111c⟩≡ (122b)
void
Xversion(Fsrpc *r)
{
    Fcall thdr;
    vlong t;

    t = nsec();

    if(r->work.msize > IOHDRSZ+Maxfdata)
        thdr.msize = IOHDRSZ+Maxfdata;
    else
        thdr.msize = r->work.msize;
    myiounit = thdr.msize - IOHDRSZ;
    if(strncmp(r->work.version, "9P2000", 6) != 0){
        reply(&r->work, &thdr, Ebadver);
        r->busy = 0;
        return;
    }
    thdr.version = "9P2000";
    /* BUG: should clunk all fids */
    reply(&r->work, &thdr, 0);
    r->busy = 0;
}

```



```

        update(&stats->rpc[Tversion], t);
    }

⟨function Xauth 112a⟩≡ (122b)
void
Xauth(Fsrpc *r)
{
    Fcall thdr;
    vlong t;

    t = nsec();

    reply(&r->work, &thdr, Enoauth);
    r->busy = 0;

    update(&stats->rpc[Tauth], t);
}

⟨function Xflush 112b⟩≡ (122b)
void
Xflush(Fsrpc *r)
{
    Fsrpc *t, *e;
    Fcall thdr;

    e = &Workq[Nr_workbufs];

    for(t = Workq; t < e; t++) {
        if(t->work.tag == r->work.older) {
            DEBUG(2, "\\tQ busy %d pid %p can %d\\n", t->busy, t->pid, t->canint);
            if(t->busy && t->pid) {
                t->flushtag = r->work.tag;
                DEBUG(2, "\\tset flushtag %d\\n", r->work.tag);
                if(t->canint)
                    postnote(PNPROC, t->pid, "flush");
                r->busy = 0;
                return;
            }
        }
    }

    reply(&r->work, &thdr, 0);
    DEBUG(2, "\\tflush reply\\n");
    r->busy = 0;
}

⟨function Xattach 112c⟩≡ (122b)
void
Xattach(Fsrpc *r)
{
    Fcall thdr;
    Fid *f;
    vlong t;

    t = nsec();

    f = newfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
    }
}

```

```

        r->busy = 0;
        return;
    }

    f->f = root;
    thdr.qid = f->f->qid;
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tattach], t);
}

```

<function Xwalk 113>≡ (122b)

```

void
Xwalk(Fsrpc *r)
{
    char errbuf[ERRMAX], *err;
    Fcall thdr;
    Fid *f, *n;
    File *nf;
    vlong t;
    int i;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }
    n = nil;
    if(r->work.newfid != r->work.fid){
        n = newfid(r->work.newfid);
        if(n == 0) {
            reply(&r->work, &thdr, Edupfid);
            r->busy = 0;
            return;
        }
        n->f = f->f;
        f = n; /* walk new guy */
    }

    thdr.nwqid = 0;
    err = nil;
    for(i=0; i<r->work.nwname; i++){
        if(i >= MAXWELEM)
            break;
        if(strcmp(r->work.wname[i], "..") == 0) {
            if(f->f->parent == 0) {
                err = Exmnt;
                break;
            }
            f->f = f->f->parent;
            thdr.wqid[thdr.nwqid++] = f->f->qid;
            continue;
        }

        nf = file(f->f, r->work.wname[i]);
        if(nf == 0) {

```

```

        errstr(errbuf, sizeof errbuf);
        err = errbuf;
        break;
    }

    f->f = nf;
    thdr.wqid[thdr.nwqid++] = nf->qid;
    continue;
}

if(err == nil && thdr.nwqid == 0 && r->work.nwname > 0)
    err = "file does not exist";

if(n != nil && (err != 0 || thdr.nwqid < r->work.nwname)){
    /* clunk the new fid, which is the one we walked */
    freefid(n->nr);
}

if(thdr.nwqid > 0)
    err = nil;
reply(&r->work, &thdr, err);
r->busy = 0;

update(&stats->rpc[Twalk], t);
}

```

<function Xclunk 114a>≡ (122b)

```

void
Xclunk(Fsrpc *r)
{
    Fcall thdr;
    Fid *f;
    vlong t;
    int fid;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }

    if(f->fid >= 0)
        close(f->fid);

    fid = r->work.fid;
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tclunk], t);

    if(f->nread || f->nwrite)
        fidreport(f);

    freefid(fid);
}

```

<function Xstat 114b>≡ (122b)

```

void
Xstat(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    uchar statbuf[STATMAX];
    Fcall thdr;
    Fid *f;
    int s;
    vlong t;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }
    makepath(path, f->f, "");
    if(!okfile(path, -1)){
        snprintf(err, sizeof err, "iostats: can't simulate %s", path);
        reply(&r->work, &thdr, err);
        r->busy = 0;
        return;
    }

    if(f->fid >= 0)
        s = fstat(f->fid, statbuf, sizeof statbuf);
    else
        s = stat(path, statbuf, sizeof statbuf);

    if(s < 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
        r->busy = 0;
        return;
    }
    thdr.stat = statbuf;
    thdr.nstat = s;
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tstat], t);
}

```

<function Xcreate 115>≡ (122b)

```

void
Xcreate(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    Fcall thdr;
    Fid *f;
    File *nf;
    vlong t;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
    }

```

```

    r->busy = 0;
    return;
}

makepath(path, f->f, r->work.name);
f->fid = create(path, r->work.mode, r->work.perm);
if(f->fid < 0) {
    errstr(err, sizeof err);
    reply(&r->work, &thdr, err);
    r->busy = 0;
    return;
}

nf = file(f->f, r->work.name);
if(nf == 0) {
    errstr(err, sizeof err);
    reply(&r->work, &thdr, err);
    r->busy = 0;
    return;
}

f->mode = r->work.mode;
f->f = nf;
thdr.iounit = myiounit;
thdr.qid = f->f->qid;
reply(&r->work, &thdr, 0);
r->busy = 0;

update(&stats->rpc[Tcreate], t);
}

```

```

⟨function Xremove 116⟩≡ (122b)
void
Xremove(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    Fcall thdr;
    Fid *f;
    vlong t;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }

    makepath(path, f->f, "");
    DEBUG(2, "\tremove: %s\n", path);
    if(remove(path) < 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
        freefid(r->work.fid);
        r->busy = 0;
        return;
    }
}

```

```

f->f->inval = 1;
if(f->fid >= 0)
    close(f->fid);
freefid(r->work.fid);

reply(&r->work, &thdr, 0);
r->busy = 0;

update(&stats->rpc[Tremove], t);
}

```

<function Xwstat 117a>≡ (122b)

```

void
Xwstat(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    Fcall thdr;
    Fid *f;
    int s;
    vlong t;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }
    if(f->fid >= 0)
        s = fwstat(f->fid, r->work.stat, r->work.nstat);
    else {
        makepath(path, f->f, "");
        s = wstat(path, r->work.stat, r->work.nstat);
    }
    if(s < 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
    }
    else
        reply(&r->work, &thdr, 0);

    r->busy = 0;
    update(&stats->rpc[Twstat], t);
}

```

<function slave 117b>≡ (122b)

```

void
slave(Fsrpc *f)
{
    int r;
    Proc *p;
    uintptr pid;
    static int nproc;

    for(;;) {
        for(p = Procllist; p; p = p->next) {
            if(p->busy == 0) {
                f->pid = p->pid;
                p->busy = 1;

```

```

        pid = (uintptr_t)rendezvous((void*)p->pid, f);
        if(pid != p->pid)
            fatal("rendezvous sync fail");
        return;
    }
}

if(++nproc > MAXPROC)
    fatal("too many procs");

r = rfork(RFPROC|RFMEM);
if(r < 0)
    fatal("rfork");

if(r == 0)
    blockingslave();

p = malloc(sizeof(Proc));
if(p == 0)
    fatal("out of memory");

p->busy = 0;
p->pid = r;
p->next = Proclist;
Proclist = p;

rendezvous((void*)p->pid, p);
}
}

```

<function blockingslave 118>≡ (122b)

```

void
blockingslave(void)
{
    Proc *m;
    uintptr_t pid;
    Fsrpc *p;
    Fcall thdr;

    notify(flushaction);

    pid = getpid();

    m = rendezvous((void*)pid, 0);

    for(;;) {
        p = rendezvous((void*)pid, (void*)pid);
        if(p == (void*)~0) /* Interrupted */
            continue;

        DEBUG(2, "\tslave: %p %F b %d p %p\n", pid, &p->work, p->busy, p->pid);
        if(p->flushtag != NOTAG)
            return;

        switch(p->work.type) {
        case Tread:
            slaveread(p);
            break;
        case Twrite:
            slavewrite(p);

```

```

        break;
case Topen:
    slaveopen(p);
    break;
default:
    reply(&p->work, &thdr, "exportfs: slave type error");
}
if(p->flushtag != NOTAG) {
    p->work.type = Tflush;
    p->work.tag = p->flushtag;
    reply(&p->work, &thdr, 0);
}
p->busy = 0;
m->busy = 0;
}
}

```

(function slaveopen 119) ≡ (122b)

```

void
slaveopen(Fsrpc *p)
{
    char err[ERRMAX], path[128];
    Fcall *work, thdr;
    Fid *f;
    vlong t;

    work = &p->work;

    t = nsec();

    f = getfid(work->fid);
    if(f == 0) {
        reply(work, &thdr, Ebadfid);
        return;
    }
    if(f->fid >= 0) {
        close(f->fid);
        f->fid = -1;
    }

    makepath(path, f->f, "");
    DEBUG(2, "\topen: %s %d\n", path, work->mode);

    p->canint = 1;
    if(p->flushtag != NOTAG)
        return;

    if(!okfile(path, work->mode)){
        snprintf(err, sizeof err, "iostats can't simulate %s", path);
        reply(work, &thdr, err);
        return;
    }

    /* There is a race here I ignore because there are no locks */
    f->fid = open(path, work->mode);
    p->canint = 0;
    if(f->fid < 0) {
        errstr(err, sizeof err);
        reply(work, &thdr, err);
        return;
    }
}

```



```

}

DEBUG(2, "\\topen: fd %d\\n", f->fid);
f->mode = work->mode;
thdr.iounit = myiounit;
thdr.qid = f->f->qid;
reply(work, &thdr, 0);

update(&stats->rpc[Topen], t);
}

⟨function slaveread 120⟩≡ (122b)
void
slaveread(Fsrpc *p)
{
    char data[Maxfdata], err[ERRMAX];
    Fcall *work, thdr;
    Fid *f;
    int n, r;
    vlong t;

    work = &p->work;

    t = nsec();

    f = getfid(work->fid);
    if(f == 0) {
        reply(work, &thdr, Ebadfid);
        return;
    }

    n = (work->count > Maxfdata) ? Maxfdata : work->count;
    p->canint = 1;
    if(p->flushtag != NOTAG)
        return;
    /* can't just call pread, since directories must update the offset */
    if(f->f->qid.type&QTDIR){
        if(work->offset != f->offset){
            if(work->offset != 0){
                snprintf(err, sizeof err, "can't seek in directory from %lld to %lld", f->offset, work->offset);
                reply(work, &thdr, err);
                return;
            }
            if(seek(f->fid, 0, 0) != 0){
                errstr(err, sizeof err);
                reply(work, &thdr, err);
                return;
            }
            f->offset = 0;
        }
        r = read(f->fid, data, n);
        if(r > 0)
            f->offset += r;
    }else
        r = pread(f->fid, data, n, work->offset);
    p->canint = 0;
    if(r < 0) {
        errstr(err, sizeof err);
        reply(work, &thdr, err);
        return;
    }
}

```

```

    }

    DEBUG(2, "\\tread: fd=%d %d bytes\\n", f->fid, r);

    thdr.data = data;
    thdr.count = r;
    stats->totread += r;
    f->nread++;
    f->bread += r;
    reply(work, &thdr, 0);

    update(&stats->rpc[Tread], t);
}

⟨function slavewrite 121a⟩≡ (122b)
void
slavewrite(Fsrpc *p)
{
    char err[ERRMAX];
    Fcall *work, thdr;
    Fid *f;
    int n;
    vlong t;

    work = &p->work;

    t = nsec();

    f = getfid(work->fid);
    if(f == 0) {
        reply(work, &thdr, Ebadfid);
        return;
    }

    n = (work->count > Maxfdata) ? Maxfdata : work->count;
    p->canint = 1;
    if(p->flushtag != NOTAG)
        return;
    n = pwrite(f->fid, work->data, n, work->offset);
    p->canint = 0;
    if(n < 0) {
        errstr(err, sizeof err);
        reply(work, &thdr, err);
        return;
    }

    DEBUG(2, "\\twrite: %d bytes fd=%d\\n", n, f->fid);

    thdr.count = n;
    f->nwrite++;
    f->bwrite += n;
    stats->totwrite += n;
    reply(work, &thdr, 0);

    update(&stats->rpc[Twrite], t);
}

⟨function reopen 121b⟩≡ (122b)
void
reopen(Fid *f)

```

```

{
    USED(f);
    fatal("reopen");
}

⟨function flushaction 122a⟩≡ (122b)
void
flushaction(void *a, char *cause)
{
    USED(a);
    if(strncmp(cause, "kill", 4) == 0)
        noted(NDFLT);

    noted(NCONT);
}

⟨iostats/statsrv.c 122b⟩≡
#include <u.h>
#include <libc.h>
#include <auth.h>
#include <fcall.h>

#include "statfs.h"

⟨global Ebadfid 110a⟩
⟨global Enotdir 110b⟩
⟨global Edupfid 110c⟩
⟨global Eopen 110d⟩
⟨global Exmnt 110e⟩
⟨global Enoauth 110f⟩
⟨global Ebadver 110g⟩

⟨function okfile 111a⟩

⟨function update 111b⟩

⟨function Xversion 111c⟩

⟨function Xauth 112a⟩

⟨function Xflush 112b⟩

⟨function Xattach 112c⟩

⟨function Xwalk 113⟩

⟨function Xclunk 114a⟩

⟨function Xstat 114b⟩

⟨function Xcreate 115⟩

⟨function Xremove 116⟩

⟨function Xwstat 117a⟩

⟨function slave 117b⟩

⟨function blockingslave 118⟩

```

⟨*function* slaveopen 119⟩

⟨*function* slaveread 120⟩

⟨*function* slavewrite 121a⟩

⟨*function* reopen 121b⟩

⟨*function* flushaction 122a⟩

Glossary

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

- [Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 6
- [Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. <https://github.com/aryx/syncweb>. cited page(s) 6
- [Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 5, 6, 16, 66
- [Pad15] Yoann Padioleau. *Principia Softwarica: The ARM Linker 5l*. 2015. cited page(s) 5, 6, 19, 20, 66
- [Pad16a] Yoann Padioleau. *Principia Softwarica: The Plan 9 Core Libraries*. 2016. cited page(s) 66
- [Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 Graphics System draw*. 2016. cited page(s) 66
- [Pad18] Yoann Padioleau. *Principia Softwarica: The Plan 9 Shell rc*. 2018. cited page(s) 66